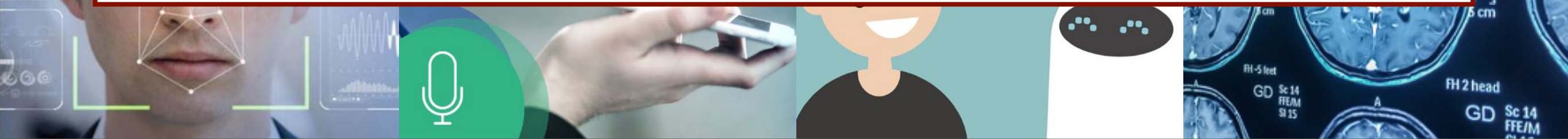


Computer Vision & Cognitive Systems

SCQ5109806 - LM CS,DS,CYB,PD



Foundations: keypoints and local visual features

Prof. Lamberto Ballan

Recall: a *(rough)* computer vision timeline

1966: Marvin Minsky assigns computer vision as an undergrad summer project

1970s: interpretation of synthetic worlds and carefully selected images

1980s: shift towards geometry and increased mathematical rigor

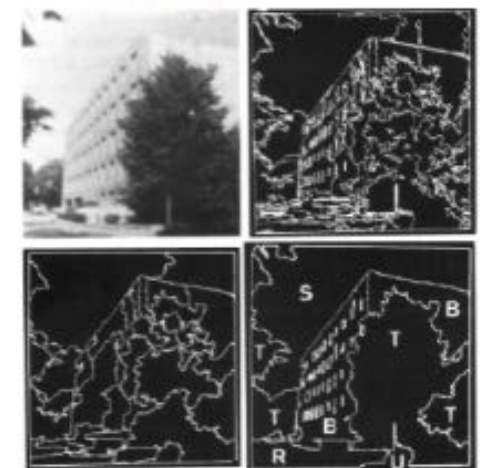
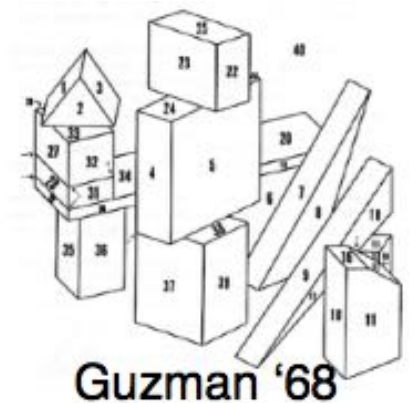
1990s: face recognition, statistical analysis

2000s: object recognition, categorization, annotated datasets available

2010s: large-scale visual recognition, visual intelligence

2020s: ???

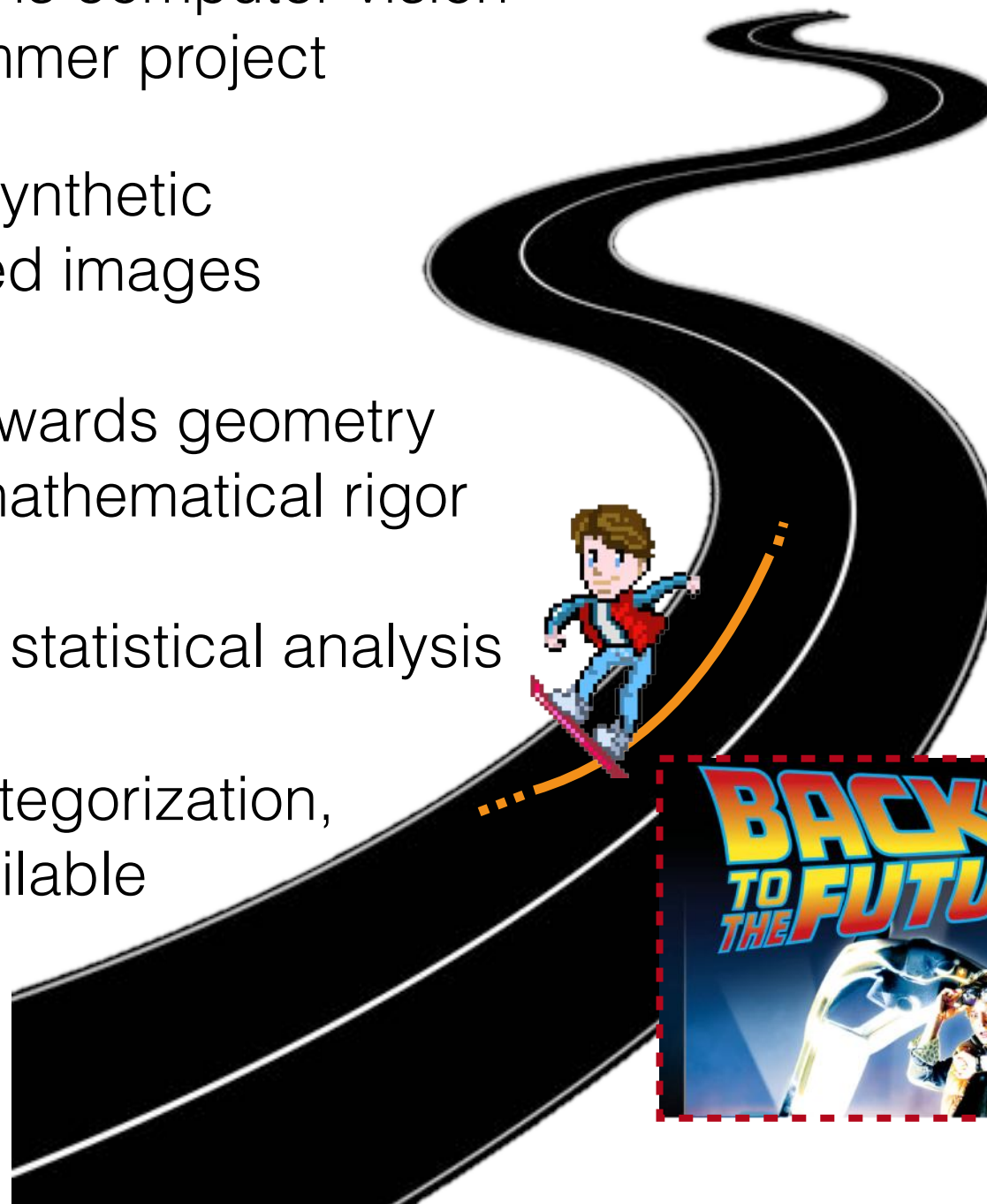
(big/“foundation” models, ...)



Ohta Kanade '78



Turk and Pentland '91



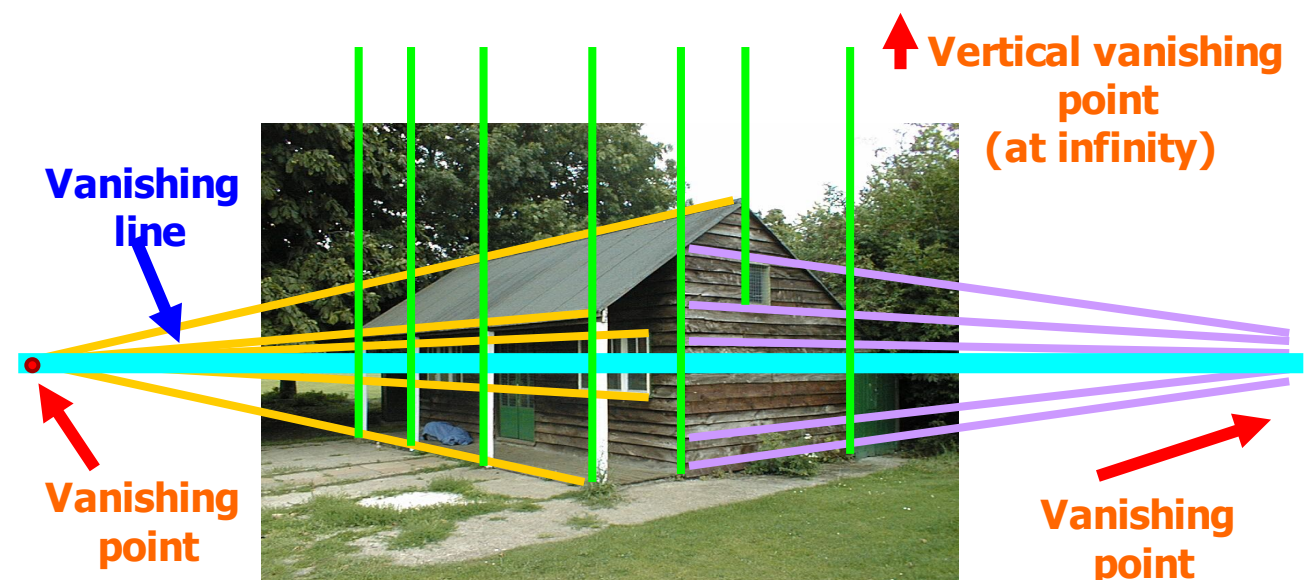
Recall: why do we care about edges?

- Extract information, recognize objects

Edges are great, but they are not very robust to a number of transformations...

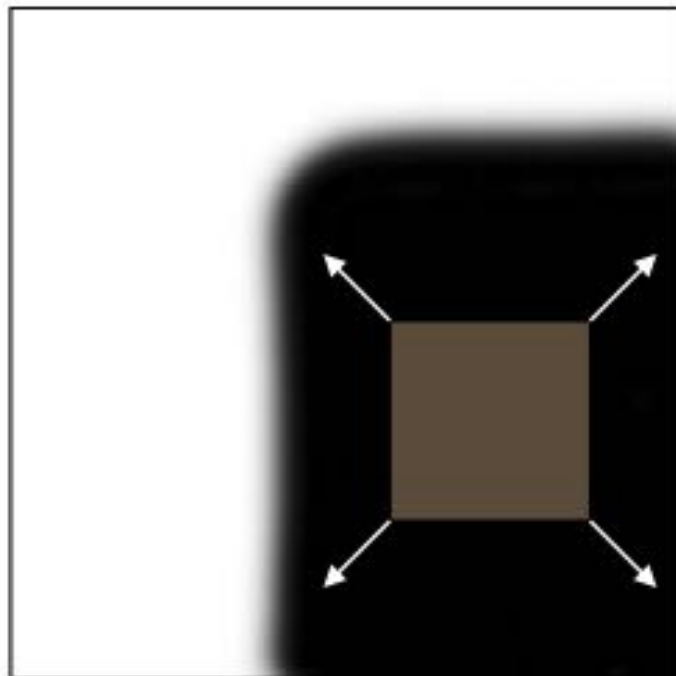


- Recover geometry and viewpoint

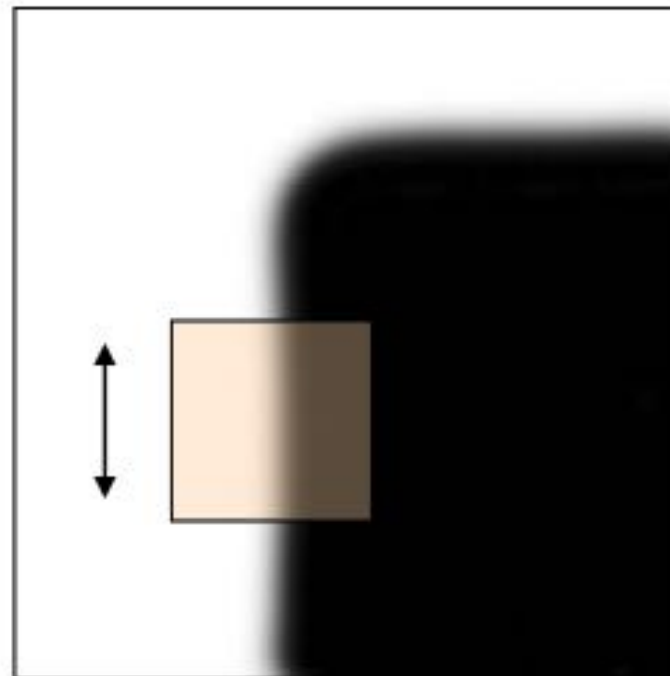


Edges, corners and blobs (keypoints)

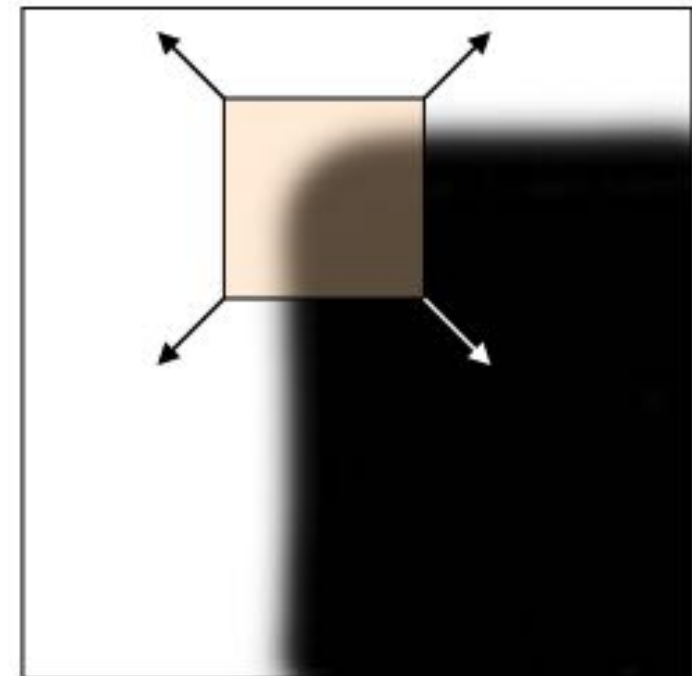
- Low-level image features: a basic classification



“flat” region:
no change in all
directions



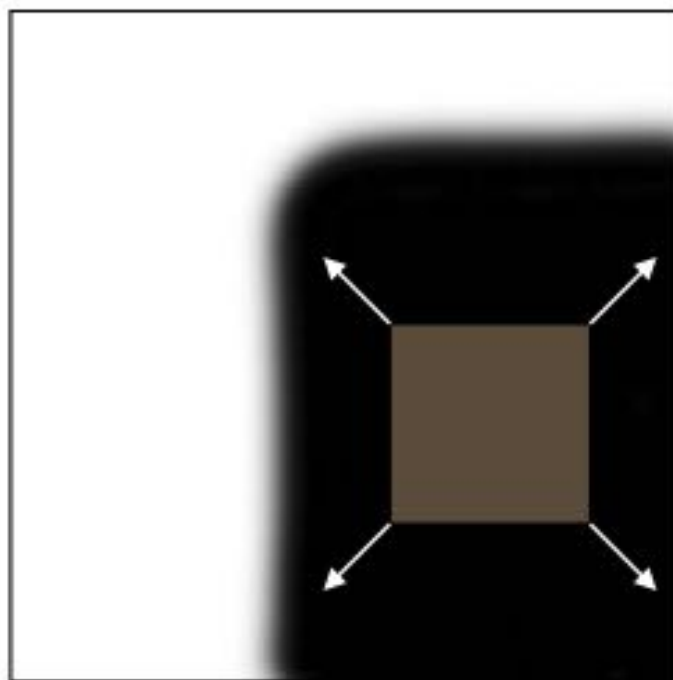
“edge”:
no change along
the edge direction



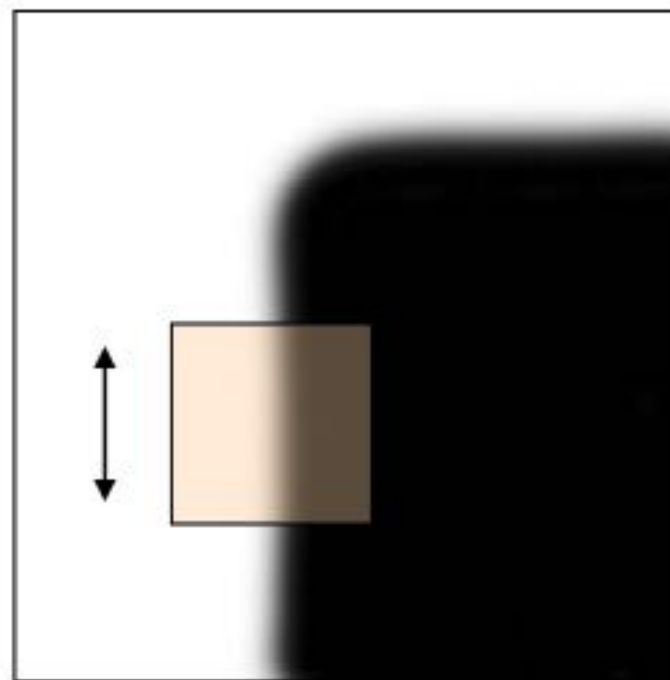
“corner”:
significant change
in all directions

Corner detection: intuition

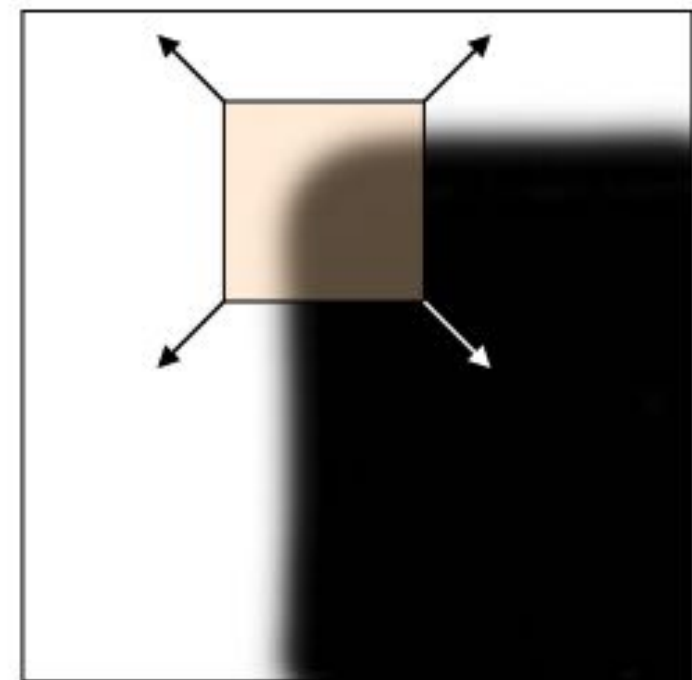
- We should easily recognize the point by looking through a small window (locally)
 - Shifting a window in any direction should give a large change in intensity



“flat” region:
no change in all
directions



“edge”:
no change along
the edge direction

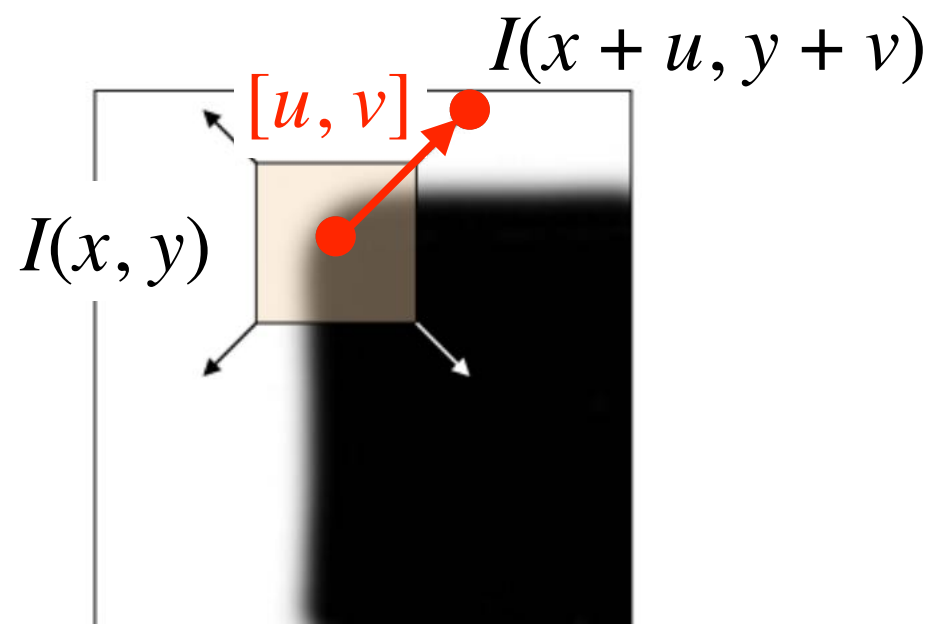


“corner”:
significant change
in all directions

Corner detection: basic idea

- Given an image I , captures change in appearance of window W for the shift $[u, v]$:

$$E(u, v) = \sum_{(x,y) \in W} [I(x + u, y + v) - I(x, y)]^2$$



“corner”:
significant change
in all directions

It measures the intensity change at the center pixel (x, y) when we shift by $[u, v]$

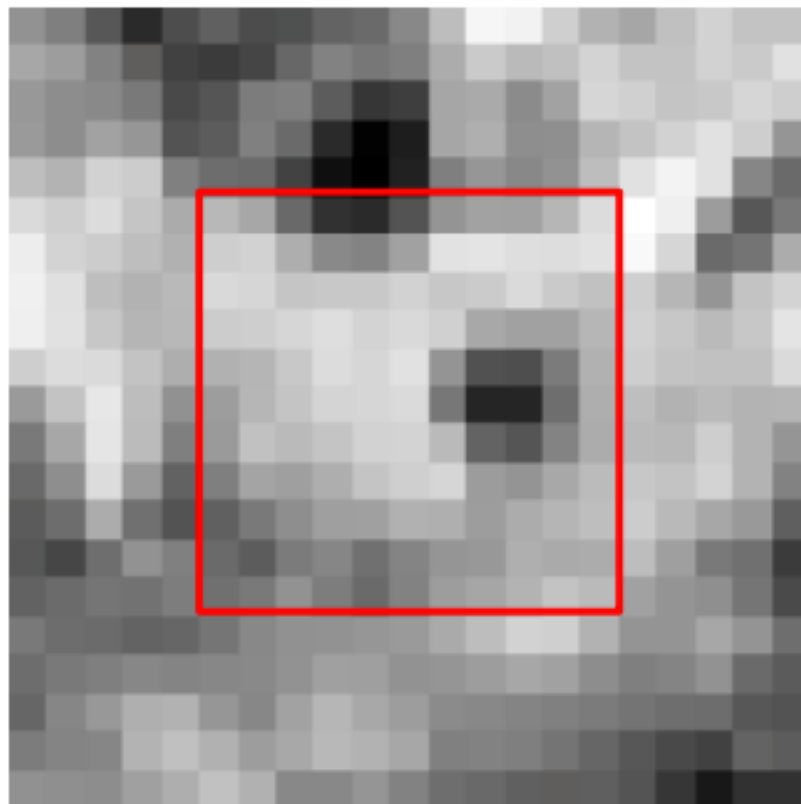
That's for a single point, $E(u, v)$ accumulates over the patch (small window) around the center pixel

Corner detection: basic idea

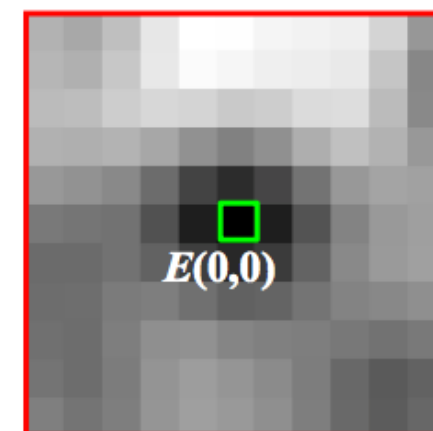
- Given an image I , captures change in appearance of window W for the shift $[u, v]$:

$$E(u, v) = \sum_{(x,y) \in W} [I(x + u, y + v) - I(x, y)]^2$$

$I(x, y)$



$E(u, v)$



Corner detection: basic idea

- Given an image I , captures change in appearance of window W for the shift $[u, v]$:

$$E(u, v) = \sum_{(x,y)} W(x, y) [I(x + u, y + v) - I(x, y)]^2$$

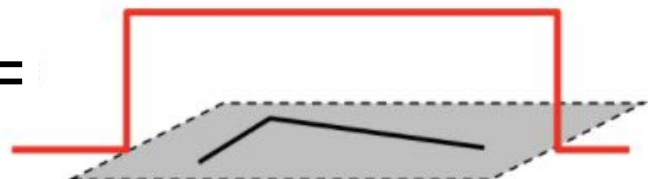
Harris Corner Detector

Window function

Intensity change

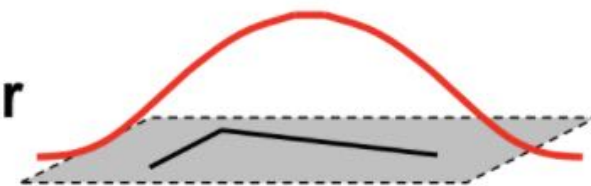
Window function

$W(x, y) =$



1 in window, 0 outside

or



Gaussian

Corners and keypoints

- Corners (e.g. Harris corner detector) are a good example of keypoints / interest points



Harris corners example

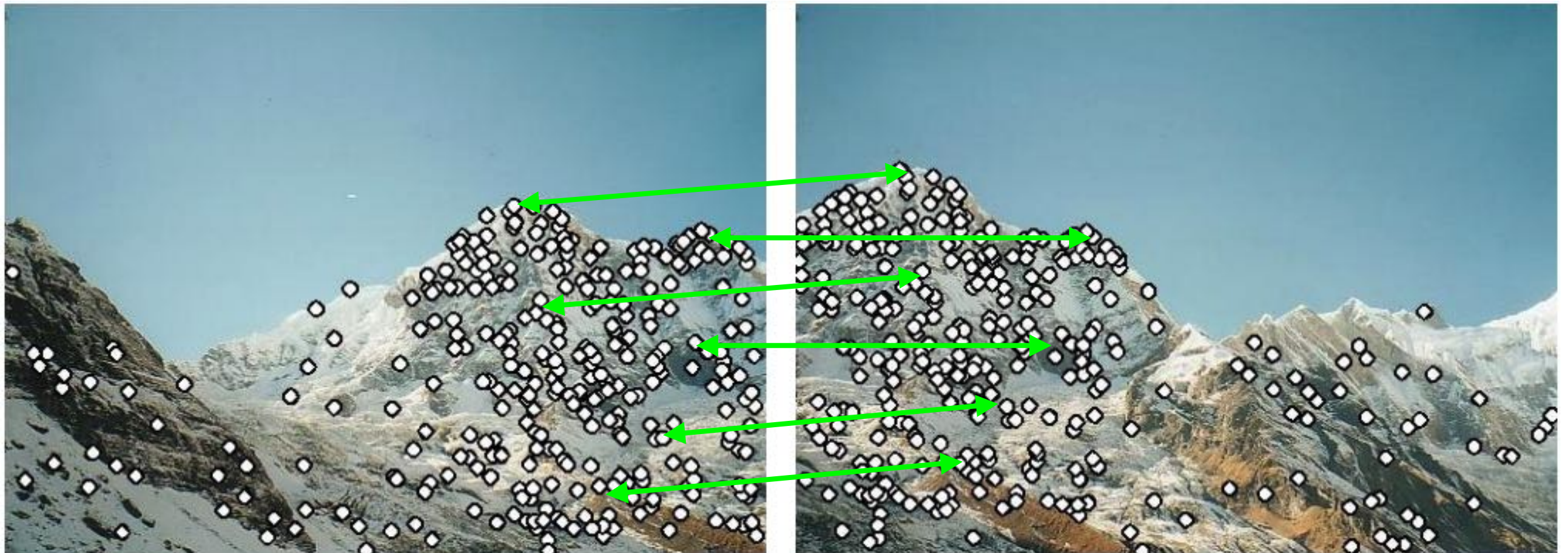
Why extract keypoints from an image?

- Motivation: a key example is panorama stitching
 - i.e. we have two images, how to combine them?



Why extract keypoints from an image?

- Motivation: a key example is panorama stitching
 - i.e. we have two images, how to combine them?



Step 1: extract keypoints

Step 2: match keypoint features

Why extract keypoints from an image?

- Motivation: a key example is panorama stitching
 - i.e. we have two images, how to combine them?



Step 1: extract keypoints

Step 2: match keypoint features

Step 3: align images

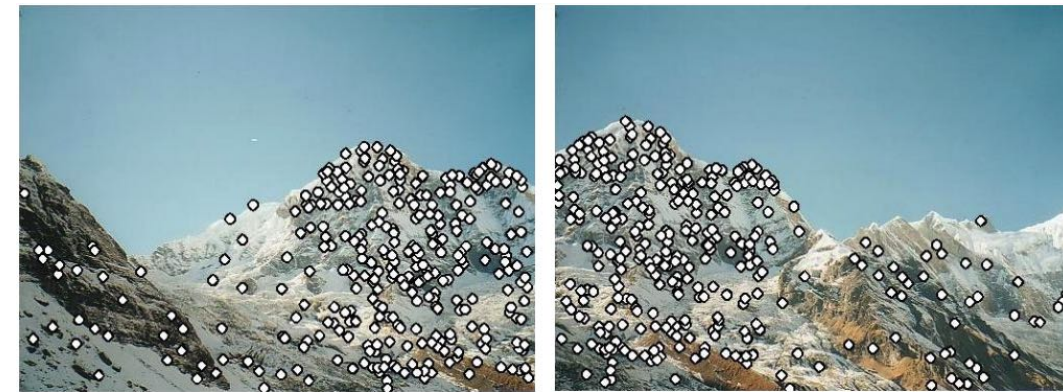
Keypoints: applications

- Keypoints are used for:
 - Image alignment
 - 3D reconstruction
 - Motion tracking
 - Robot navigation
 - Indexing and database retrieval
 - Object recognition



Characteristics of Good Keypoints

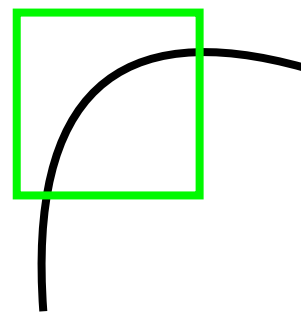
- Repeatability
 - Can be found despite geometric and photometric transformations
- Saliency
 - Each keypoint is distinctive
- Compactness and efficiency
 - Many fewer keypoints than image pixels
- Locality
 - Occupies small area of the image: therefore more robust to clutter and occlusion



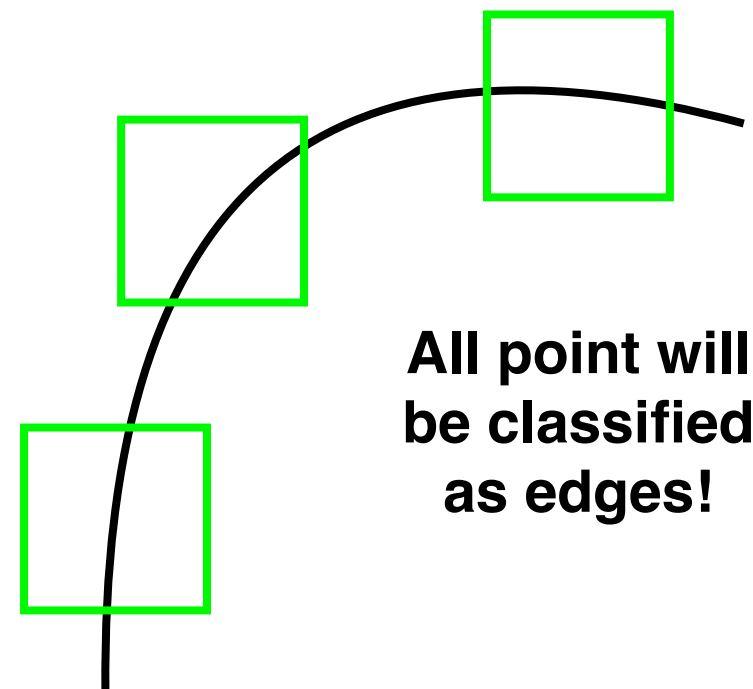
An example: Harris corner detector

- Properties (i.e. geometric invariances):

- Translation invariance? **Yes**
- Rotation invariance? **Yes**
- Scale invariance? **No**



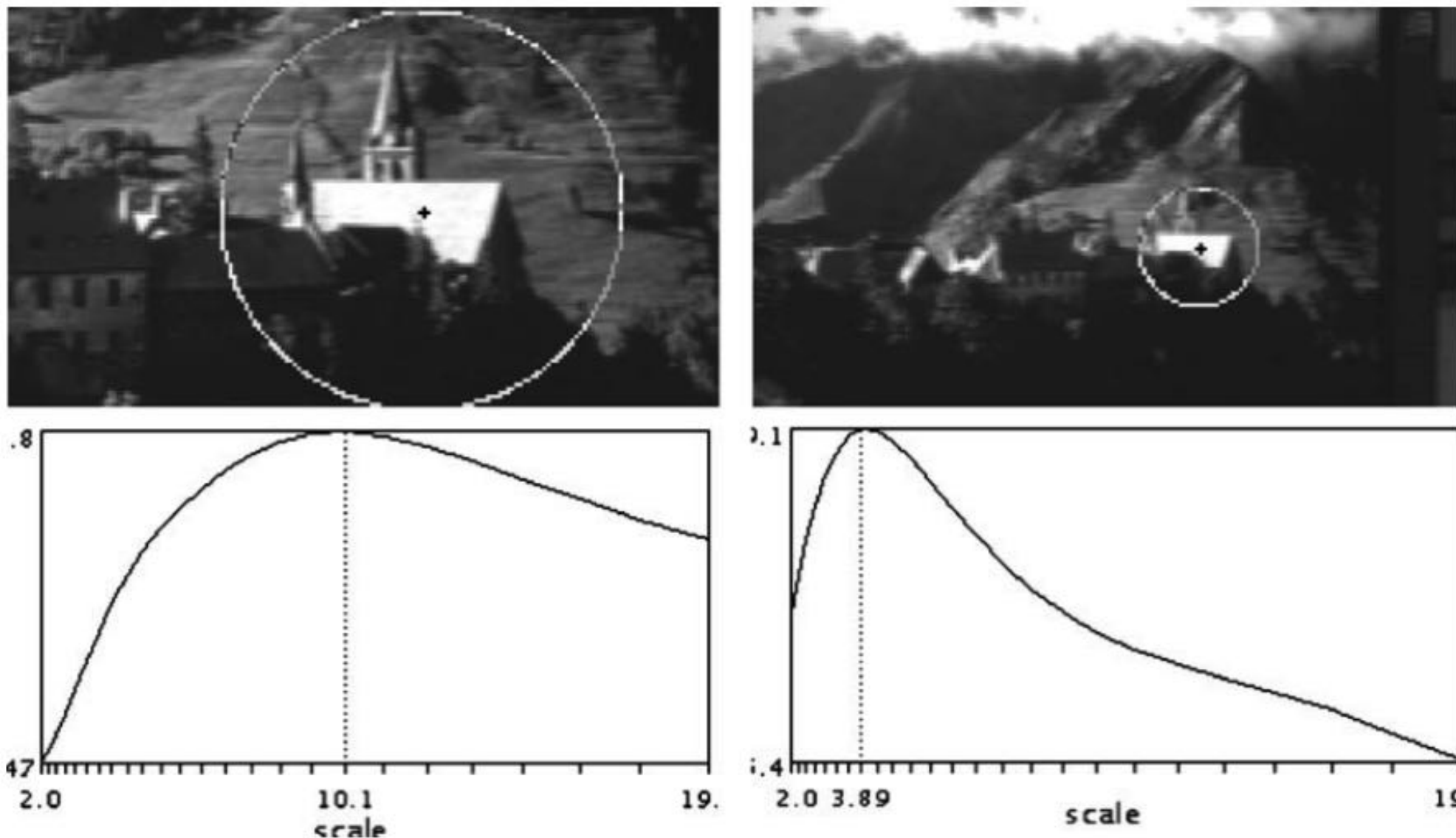
Corner



**All point will
be classified
as edges!**

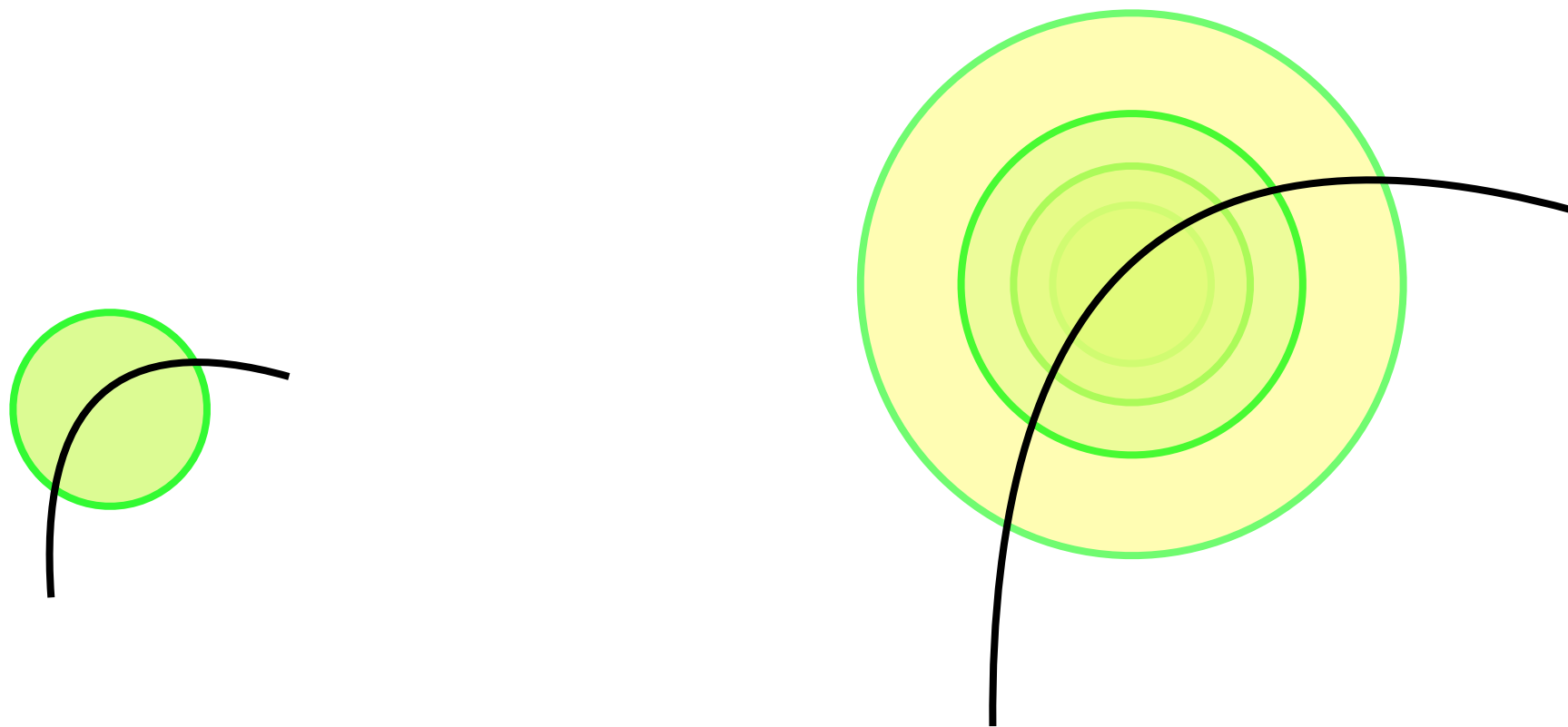
Feature detection with scale selection

- We want to extract features with characteristic scale that is invariant (*covariant*) with the image transformation



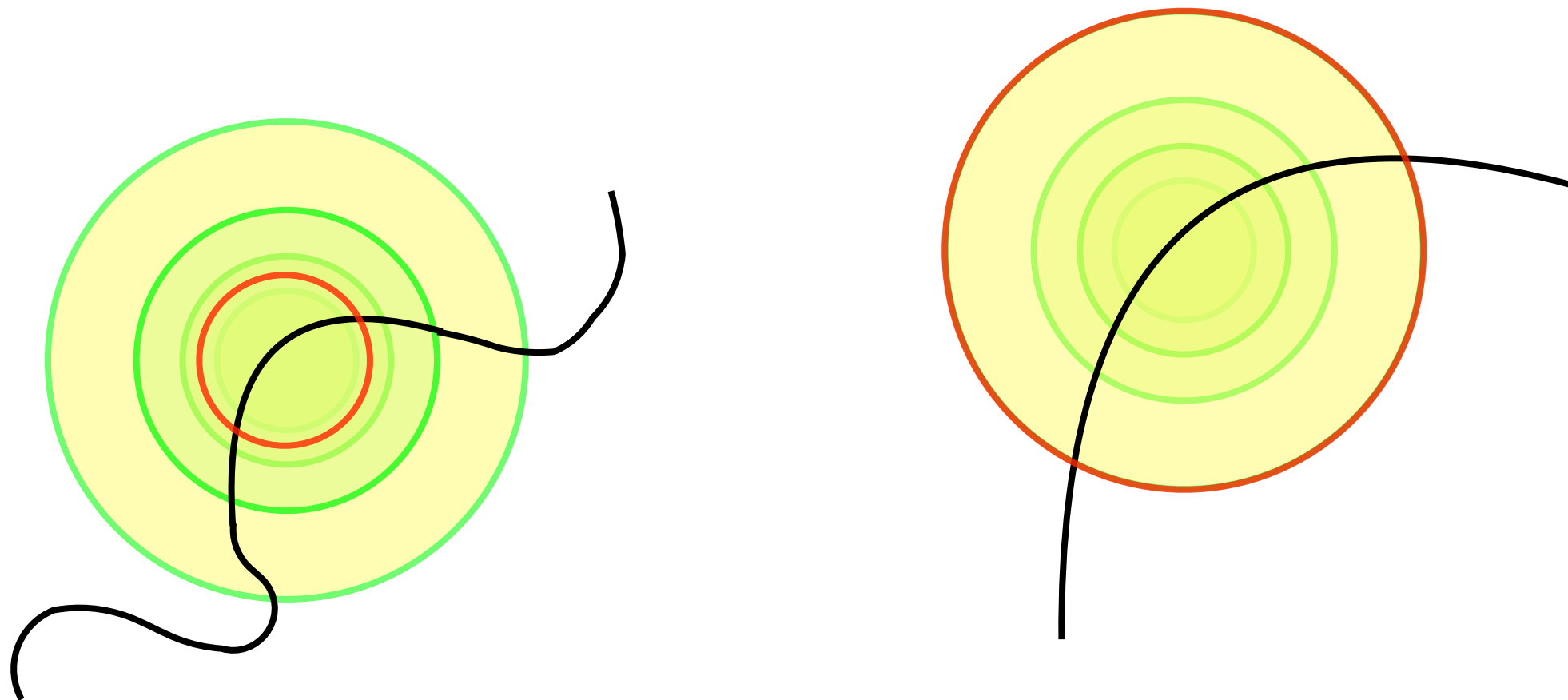
Scale invariant detection

- Consider regions (e.g. circles) of different sizes around a point
- How to find regions of corresponding sizes that will look the same in both images?



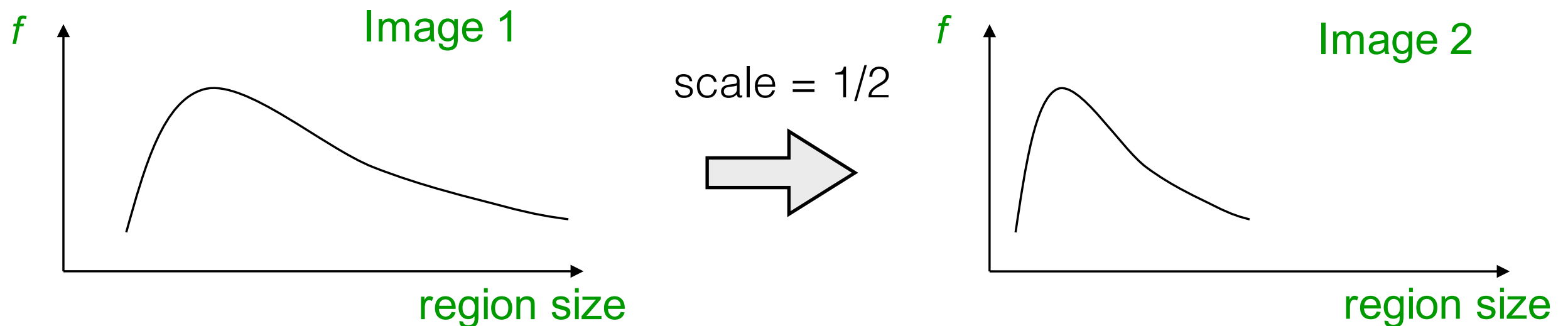
Scale invariant detection

- The problem: how do we choose corresponding circles **independently** in each image?



Scale invariant detection

- Solution: design a function on the circular region which is scale invariant *
 - e.g. average intensity (same even for different sizes)
- For a point in one image, we can consider it as a function of region size (circle radius)

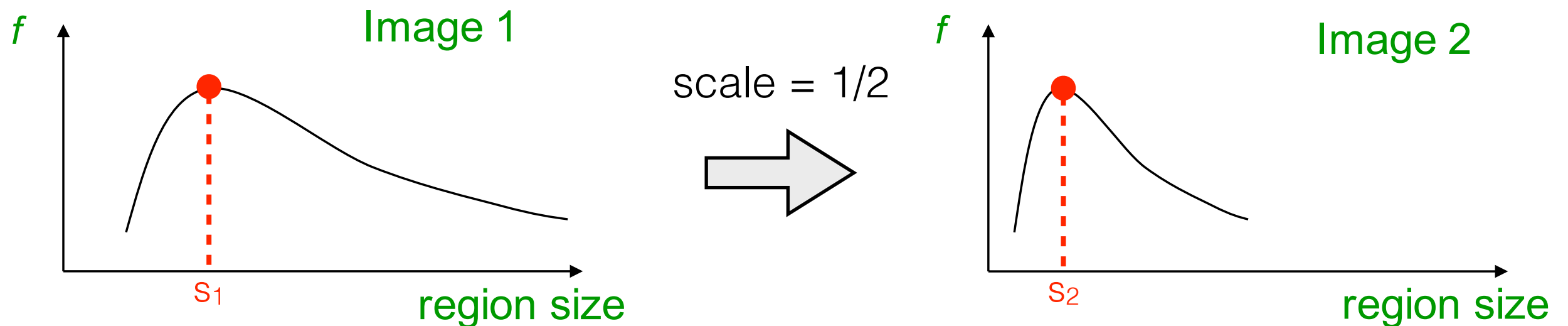


* i.e. the same for corresponding regions, even if they are at different scales

Scale invariant detection

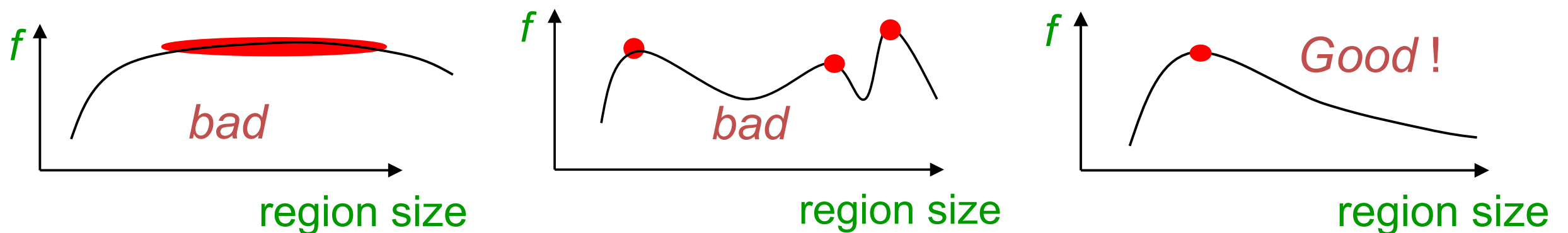
- Common approach: take a local max of this function
- Observation: region size for which the maximum is achieved should be *invariant* (*co-variant*) with scale

Important: this scale invariant region size is found in each image **independently!**



Scale invariant detection

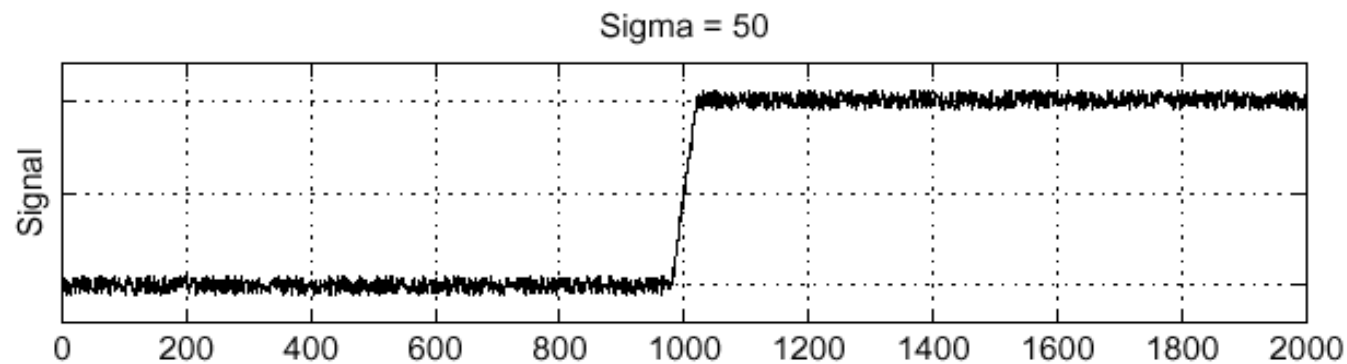
- A good function for scale detection has one stable sharp peak



- So a good function would be one which responds to contrast (sharp local intensity change)

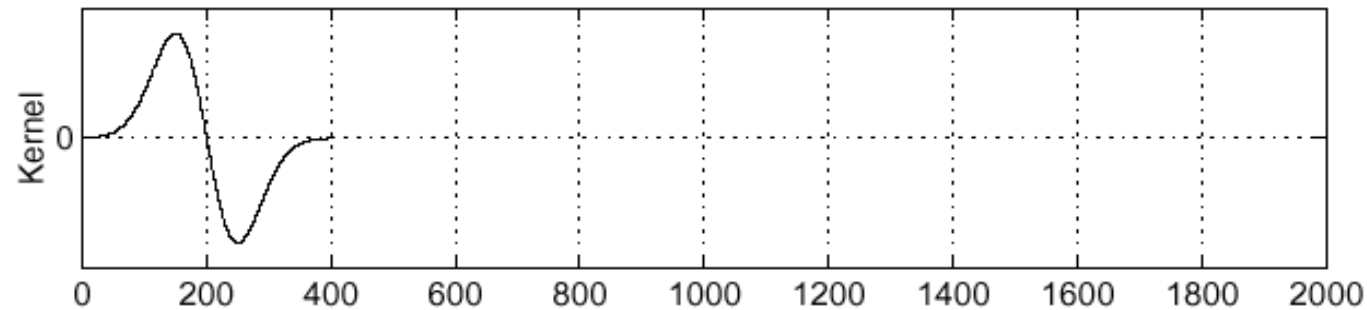
Recall: edge detection

f



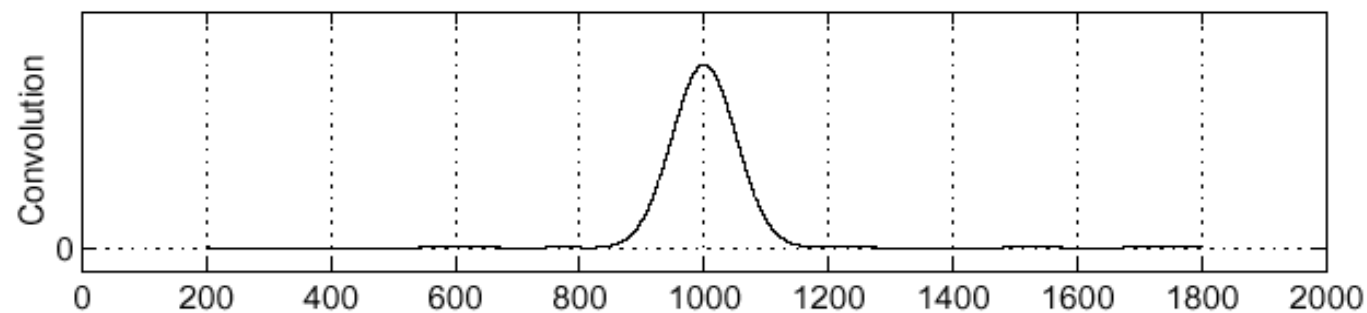
Edge

$\frac{d}{dx} g$



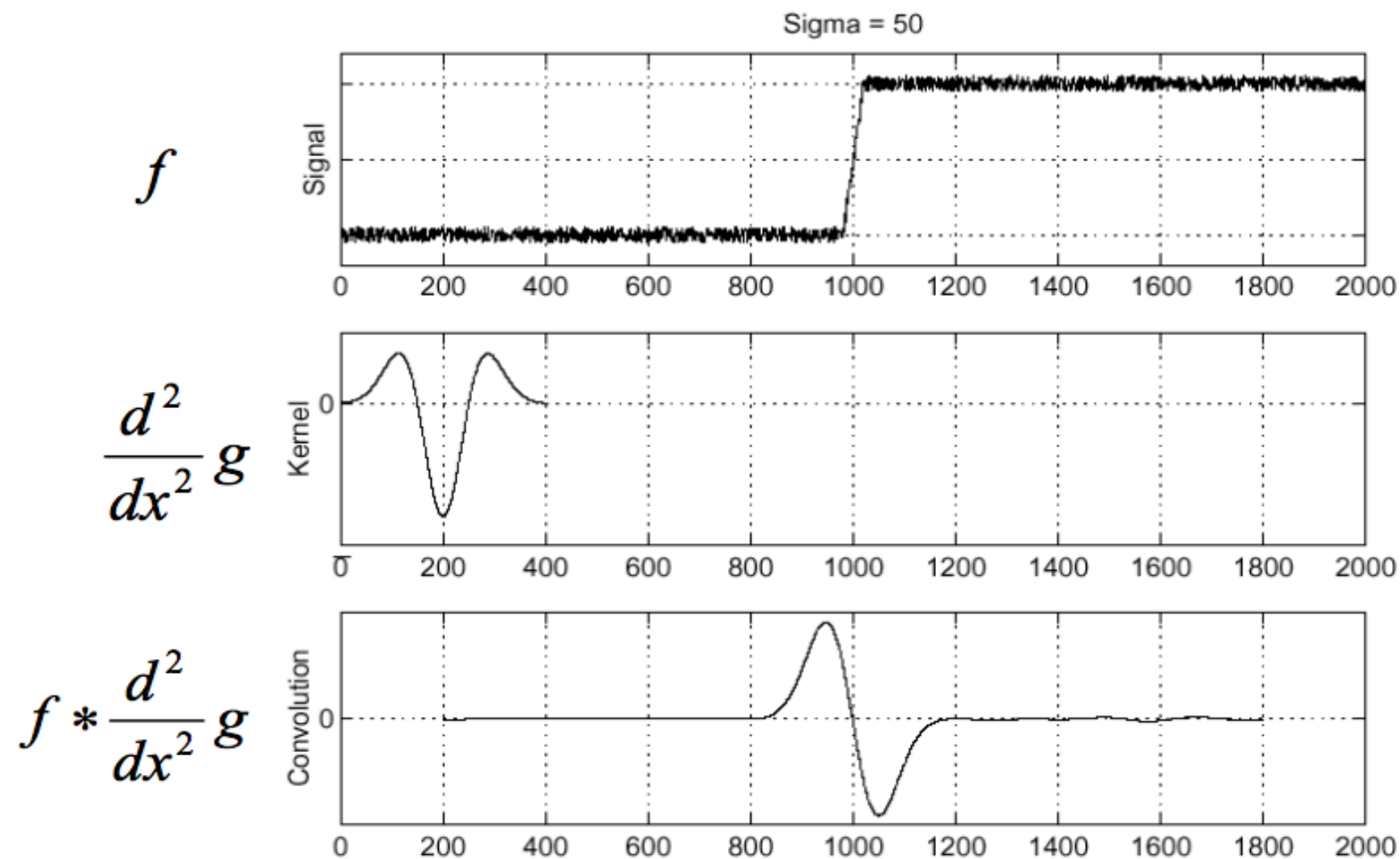
Derivative of Gaussian

$f * \frac{d}{dx} g$



Edge = maximum of derivative

A quick “detour” on edge detection



Edge

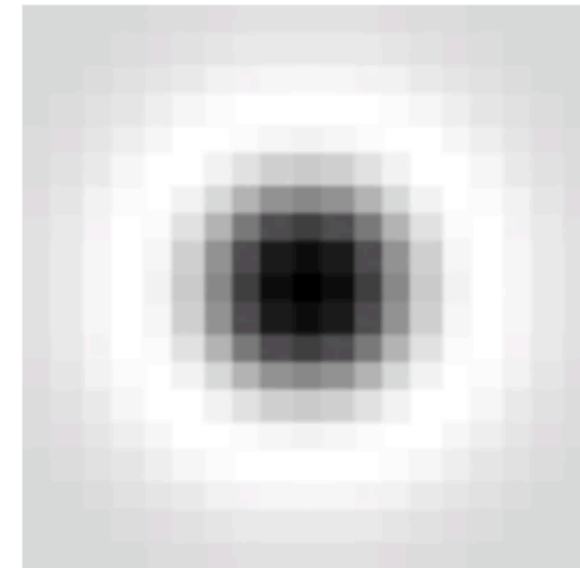
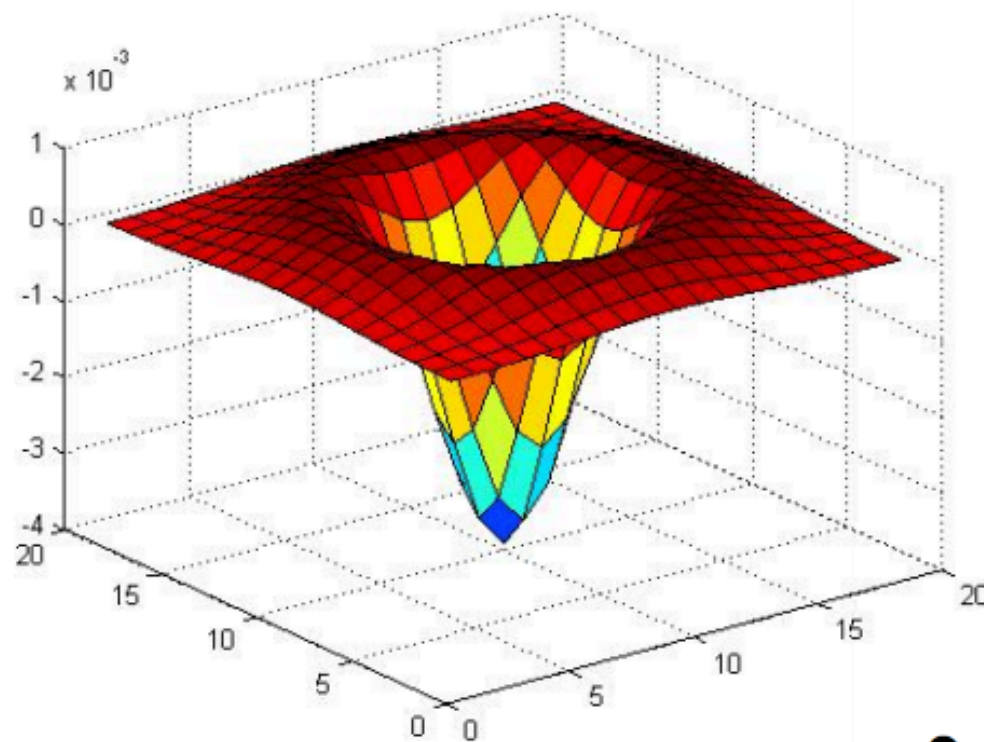
**Second Derivative of
Gaussian (Laplacian)**

Mexican hat filter

**Edge = zero crossing of
second derivative**

Blob filter

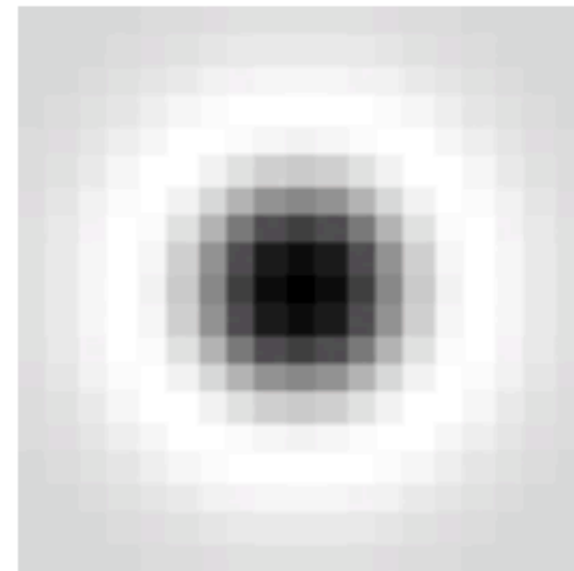
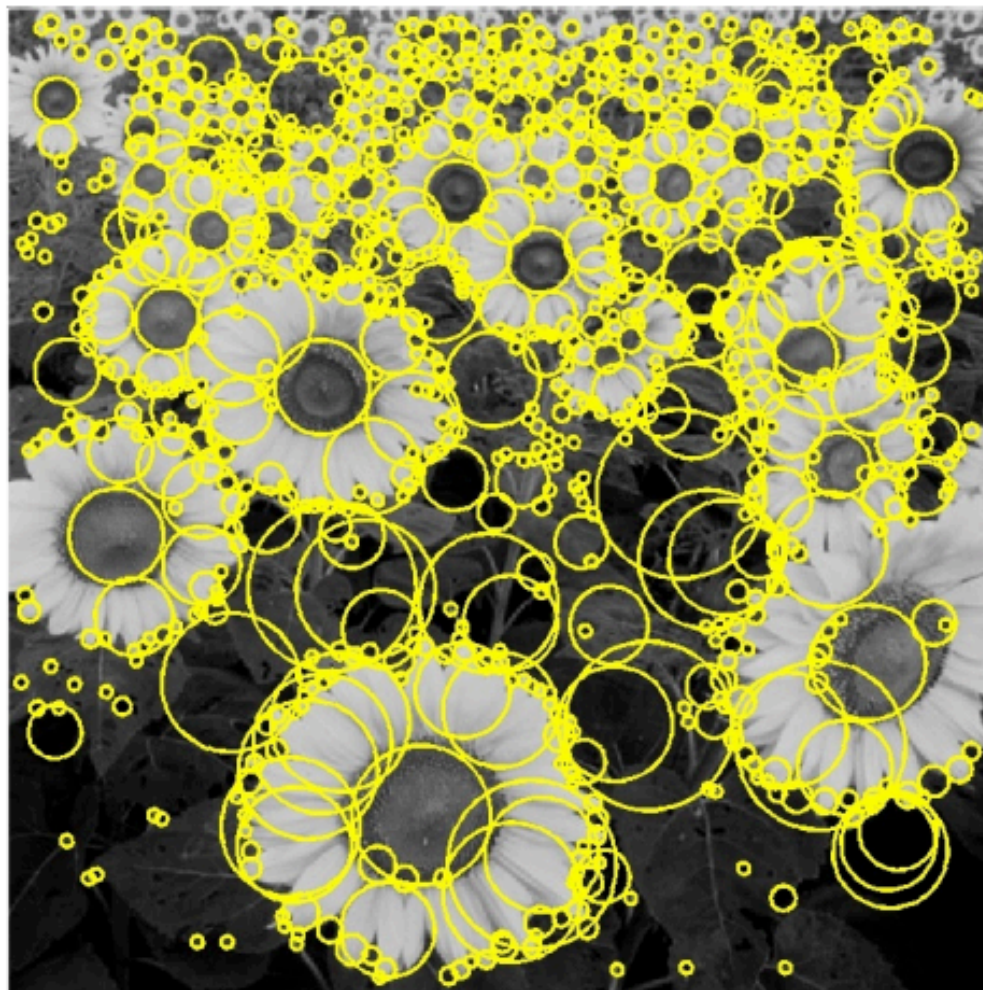
- **Laplacian of Gaussian (LoG):** circularly symmetric operator for blob detection in 2D



$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}$$

Blob detection: key idea

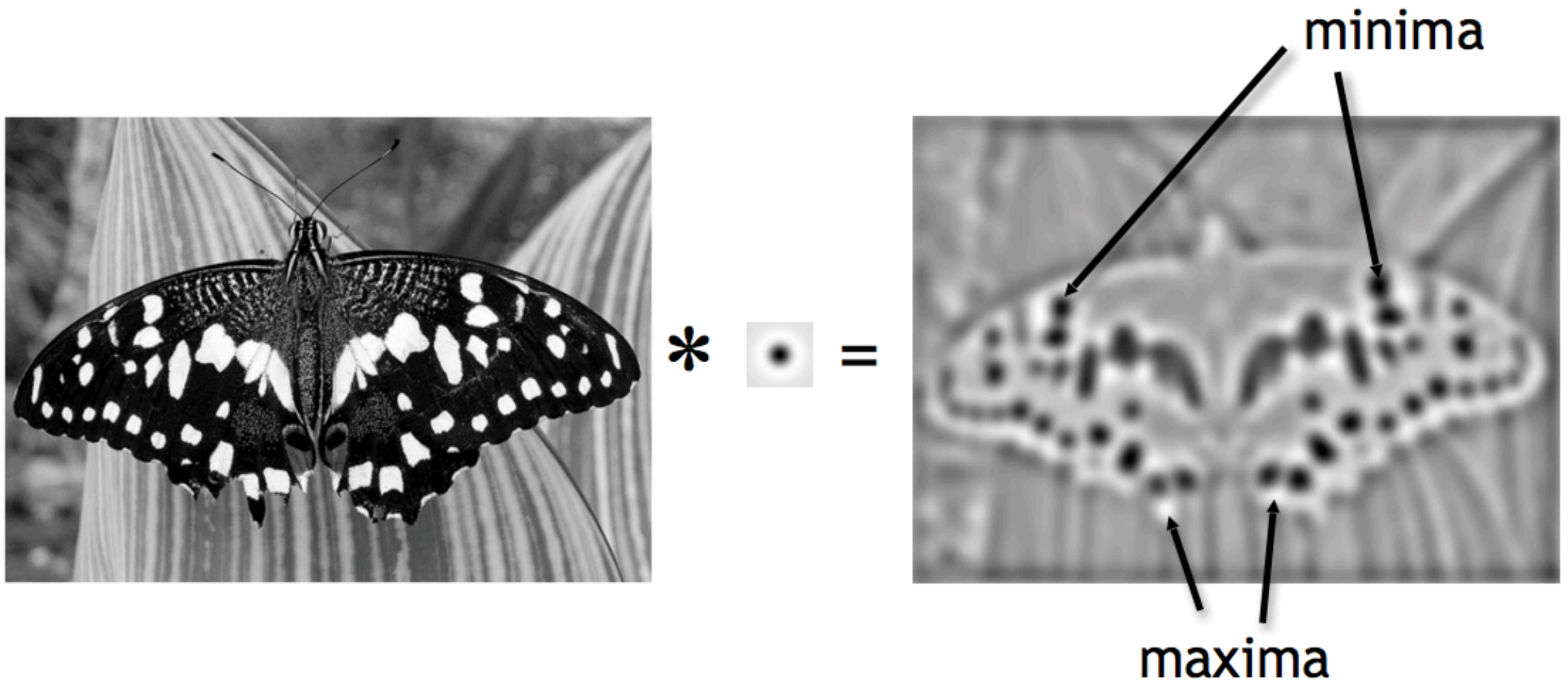
- To detect “blobs”, convolve the image with a blob filter at multiple scales and look for extrema of filter response in the resulting *scale space*



Blob filter

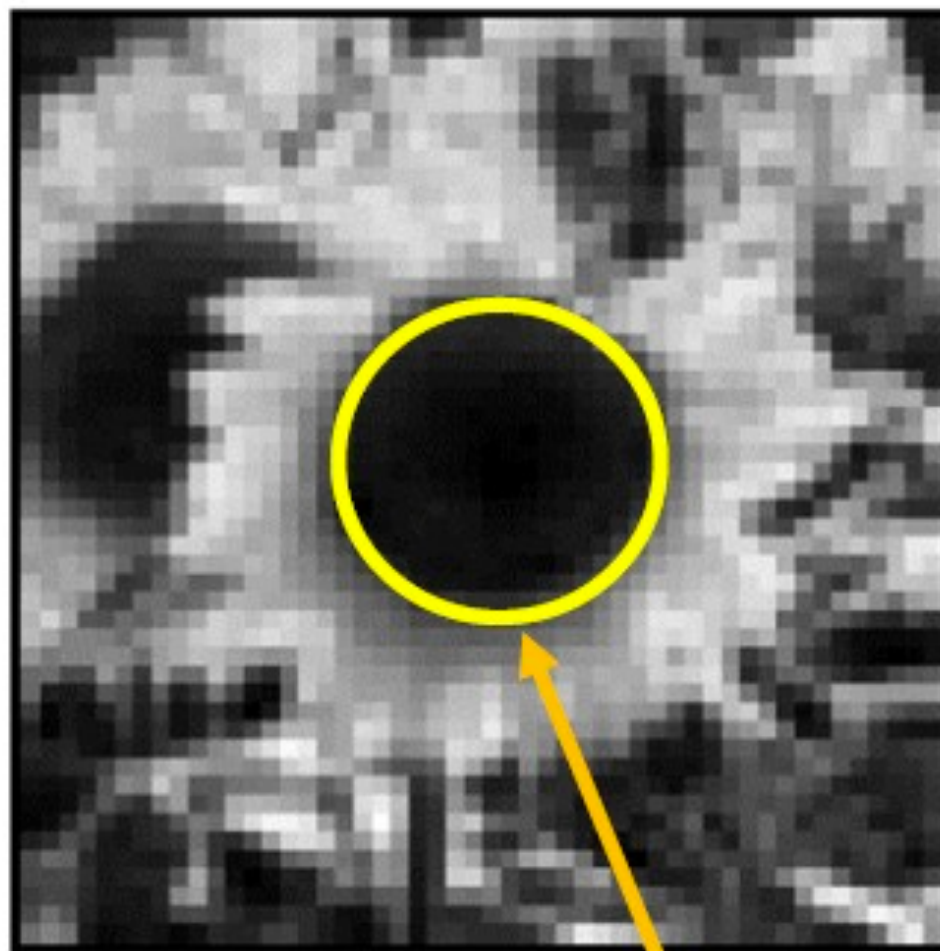
Blob detection: key idea

- Find maxima and minima of blob filter response in space and scale

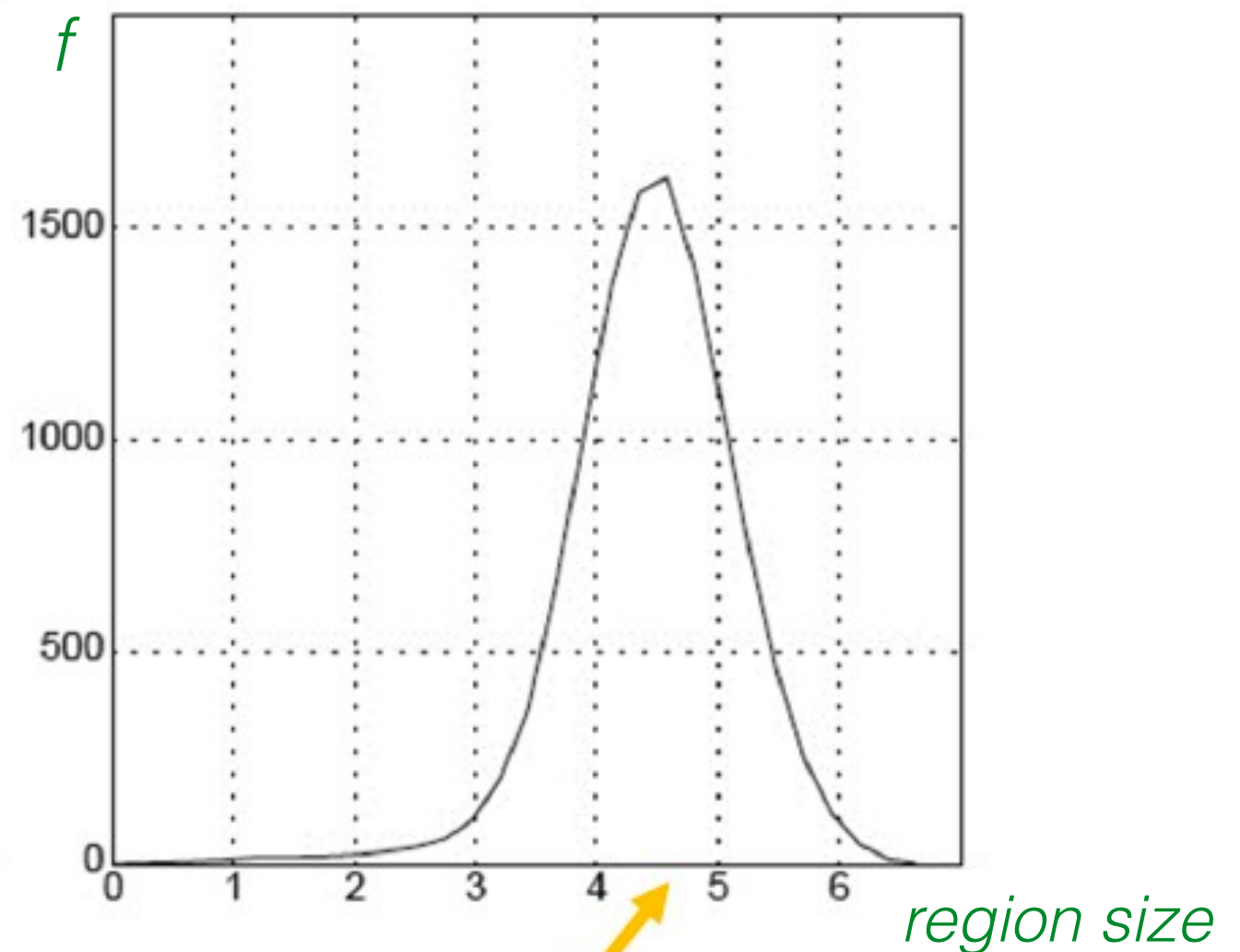


Scale invariant detection

- An example (Laplacian):

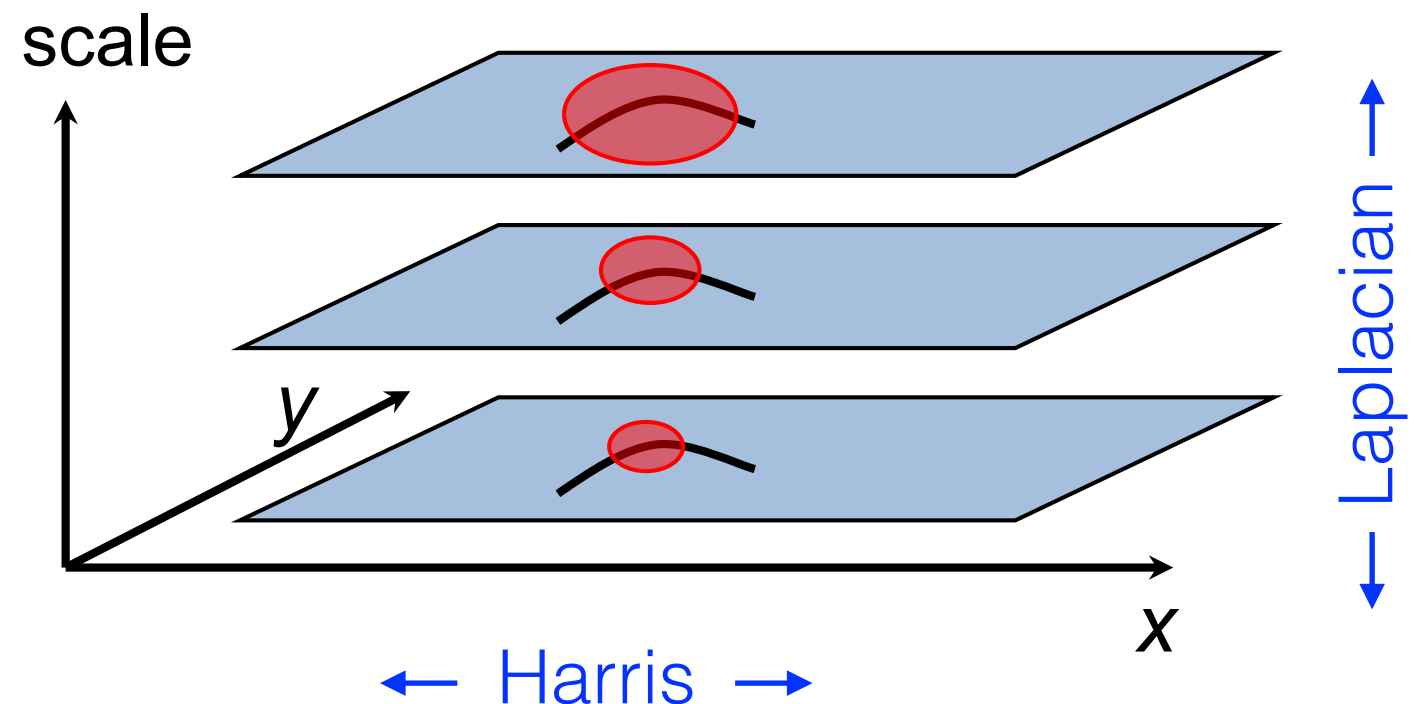


Characteristic scale



Scale invariant detection

- **Harris-Laplacian** detector: find local maximum of
 - Harris corner detector in space (image coordinates)
 - Laplacian in scale



K.Mikolajczyk, C.Schmid, "Indexing Based on Scale Invariant Interest Points", ICCV 2001

Scale invariant detection

- Functions for determining scale: $f = \text{Kernel} * \text{Image}$

Kernels:

$$L = \sigma^2 (G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma))$$

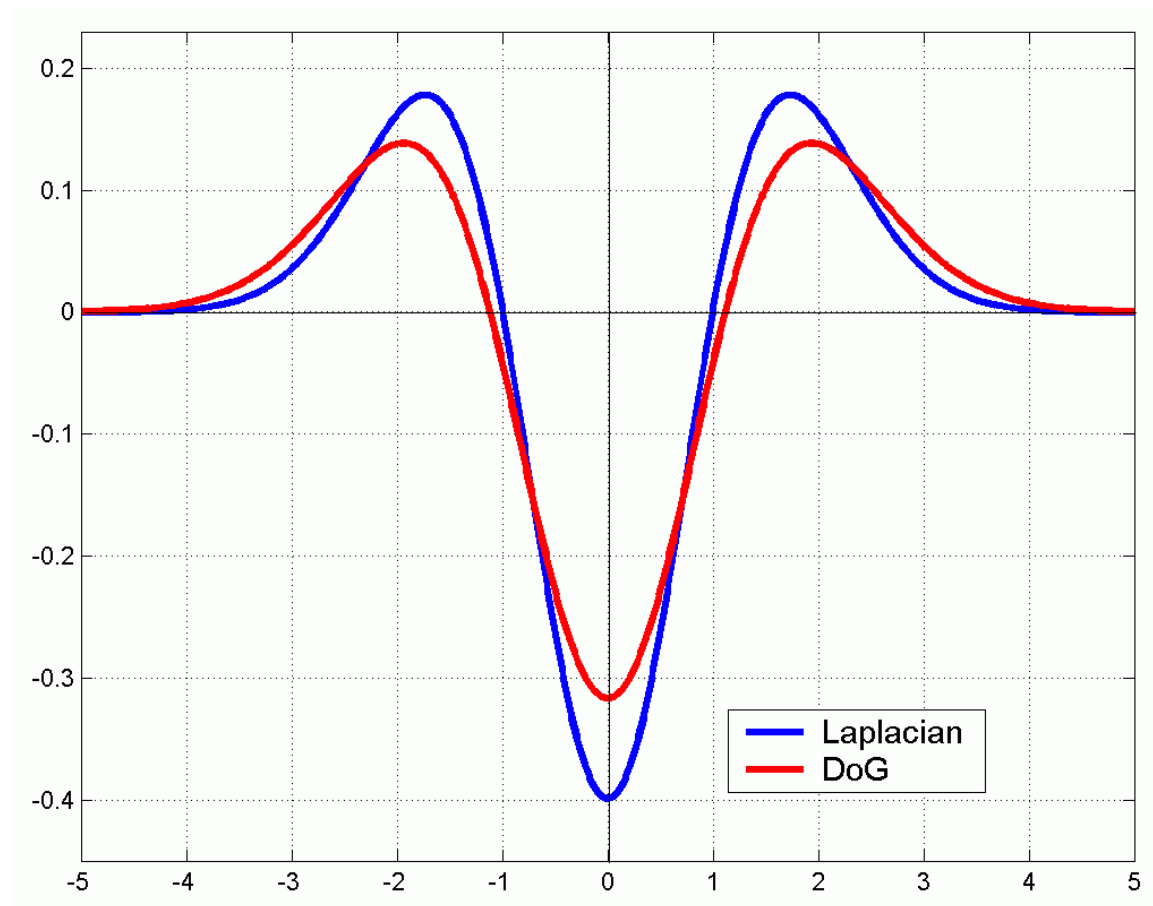
(Laplacian)

$$DoG = G(x, y, k\sigma) - G(x, y, \sigma)$$

(Difference of Gaussians)

where Gaussian

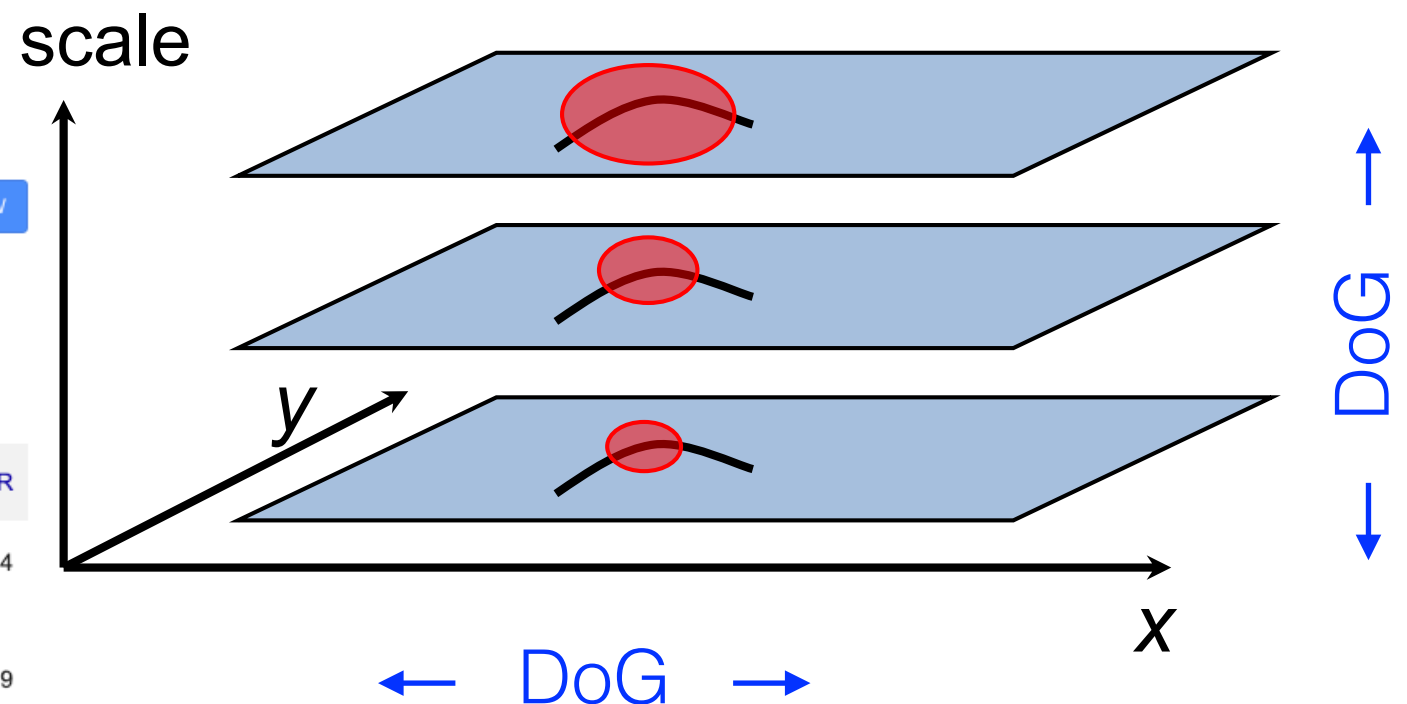
$$G(x, y, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2 + y^2}{2\sigma^2}}$$



Note: both kernels are invariant to scale and rotation

Scale invariant detection

- **SIFT** detector: find local maximum of
 - Difference of Gaussians in space
 - Difference of Gaussians in scale



David Lowe

Professor Emeritus, Computer Science Dept.,
[University of British Columbia](#)
Verified email at cs.ubc.ca - [Homepage](#)
[Computer Vision](#) [Object Recognition](#)

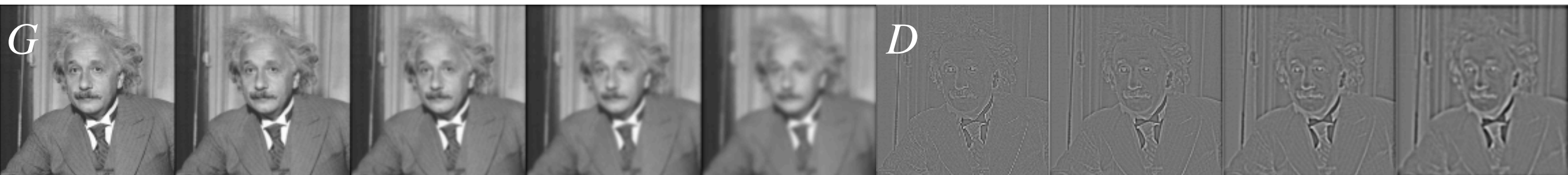
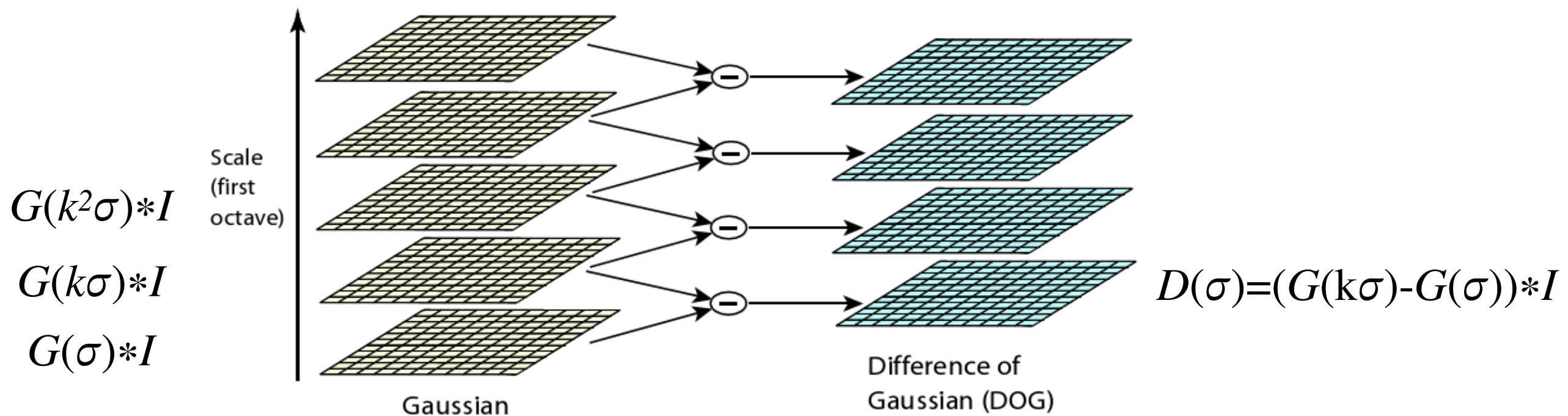


TITLE	CITED BY	YEAR
Distinctive image features from scale-invariant keypoints DG Lowe International journal of computer vision 60 (2), 91-110	72551	2004
Object recognition from local scale-invariant features DG Lowe International Conference on Computer Vision, 1999, 1150-1157	24829	1999

D. Lowe, "Distinctive image features from scale-invariant keypoints", IJCV 2004

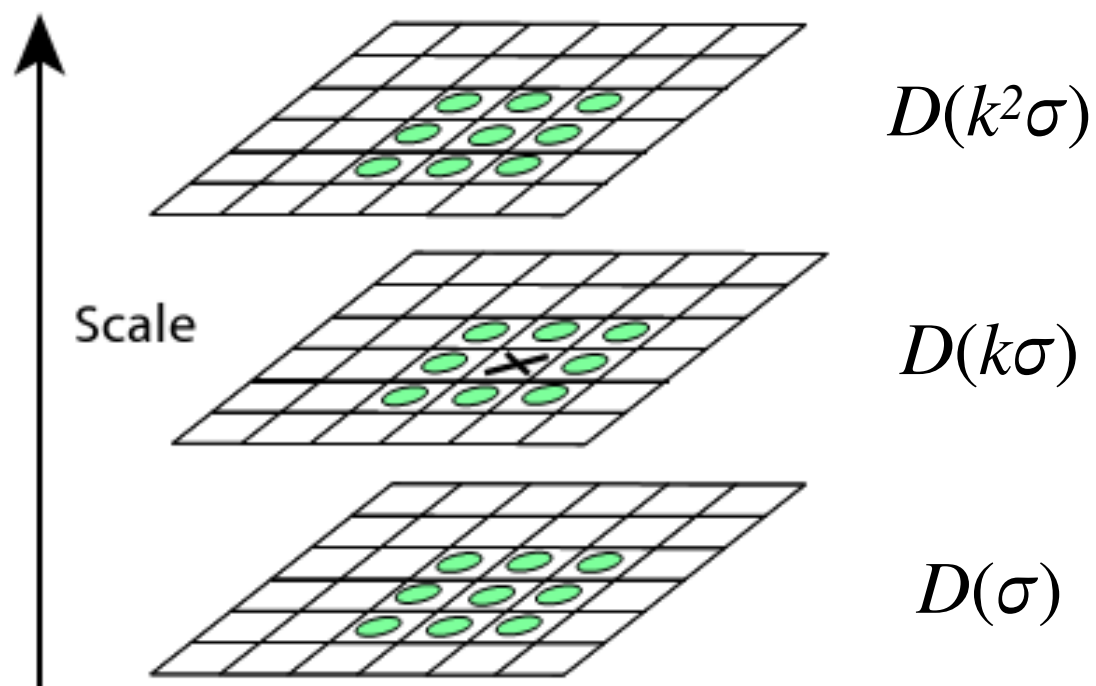
Difference of Gaussians (SIFT detector)

- Efficient approximation of LoG



Difference of Gaussians (SIFT detector)

- Keypoint are selected as scale-space extrema
 - Choose all extrema within 3x3x3 neighborhood



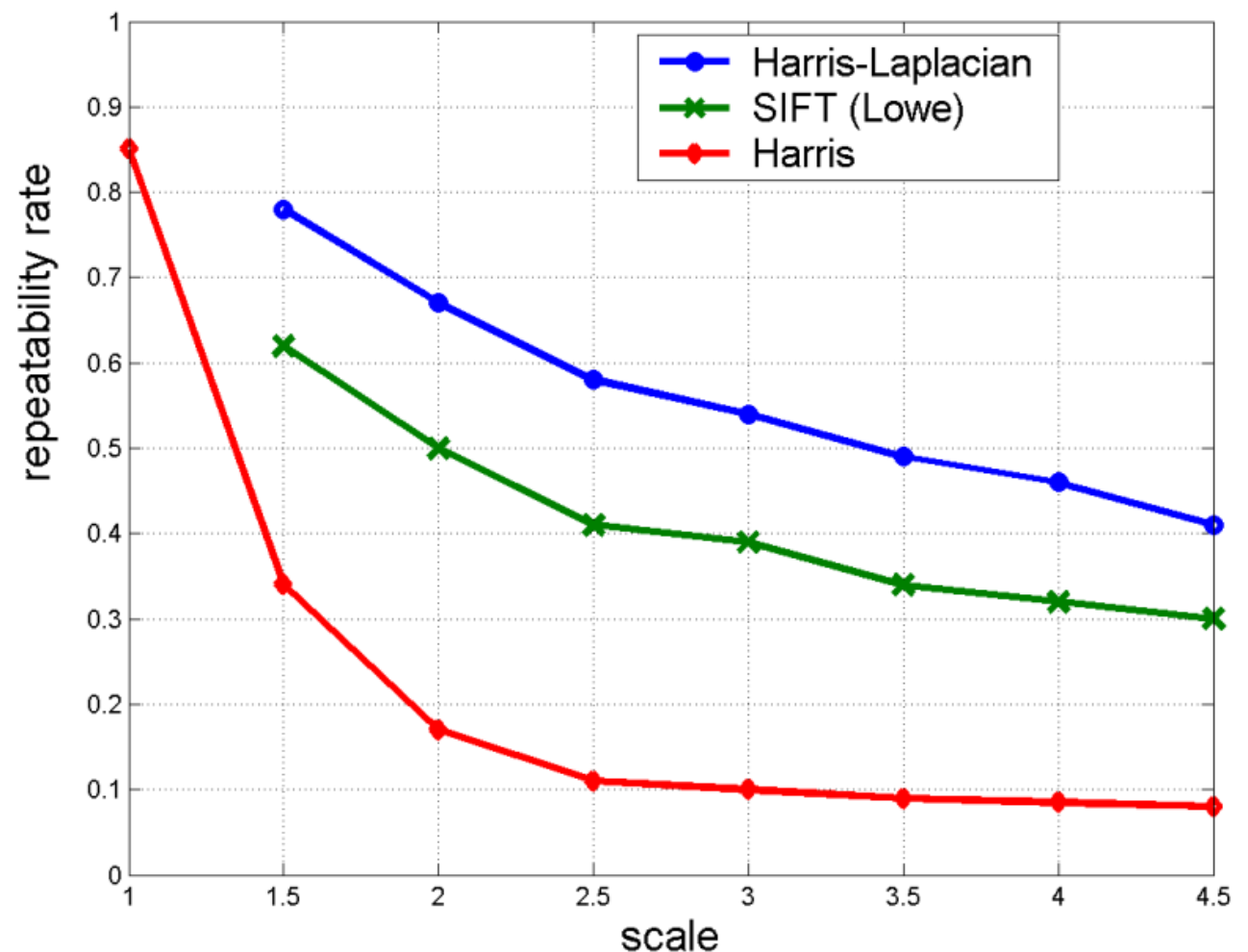
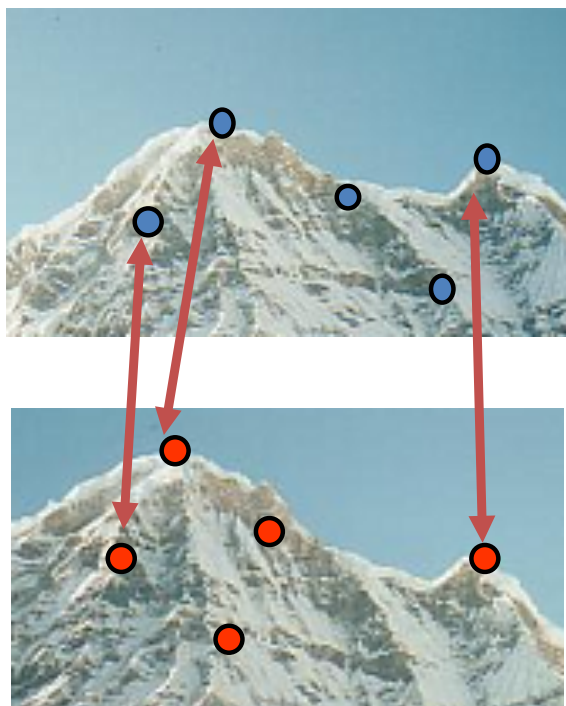
X is selected if it is larger or smaller than all 26 neighbors

Scale Invariant Detectors

- Experimental evaluation of detectors w.r.t. scale change

Repeatability rate:

$$\frac{\# \text{ correspondences}}{\# \text{ possible correspondences}}$$



K. Mikolajczyk, C. Schmid, "Indexing Based on Scale Invariant Interest Points", ICCV 2001

Scale Invariant Detectors

- Scale invariant detection summary:
 - **Given:** two images of the same scene with a large scale difference between them
 - **Goal:** find the same interest points independently in each image
 - **Solution:** search for maxima of suitable functions in scale and in space (over the image)

Popular methods:

1. Harris-Laplacian [Mikolajczyk, Schmid]: maximize Laplacian of Gaussian over scale, Harris' measure of corner response over the image
2. SIFT [Lowe]: maximize Difference of Gaussians over scale and space

Scale Invariant Detectors

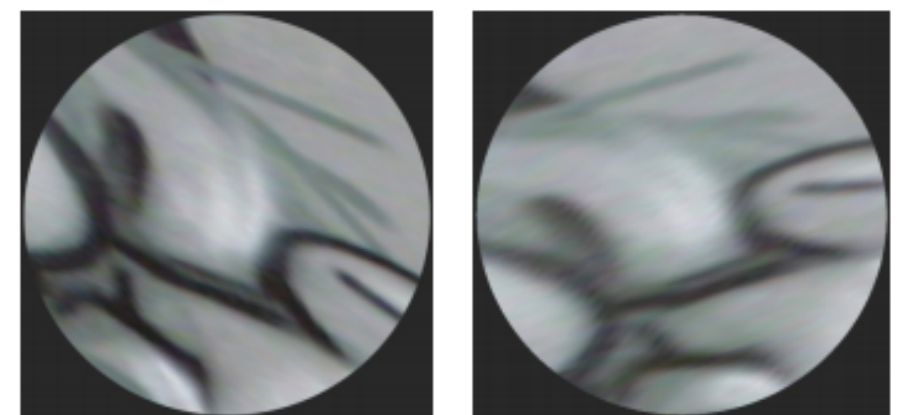
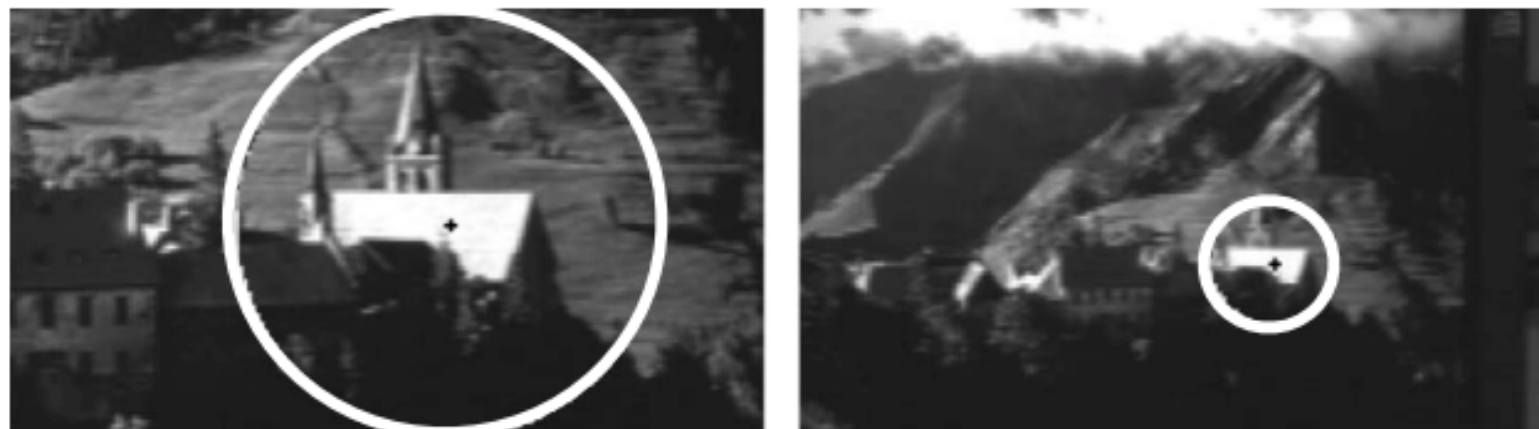
- Actually there are many other keypoints/features...
- List/comparison of keypoint detectors:

Feature Detector	Corner	Blob	Region	Rotation invariant	Scale invariant	Affine invariant	Repeatability	Localization accuracy	Robustness	Efficiency
Harris	✓			✓			+++	+++	+++	++
Hessian		✓		✓			++	++	++	+
SUSAN	✓			✓			++	++	++	+++
Harris-Laplace	✓	(✓)		✓	✓		+++	+++	++	+
Hessian-Laplace	(✓)	✓		✓	✓		+++	+++	+++	+
DoG	(✓)	✓		✓	✓		++	++	++	++
SURF	(✓)	✓		✓	✓		++	++	++	+++
Harris-Affine	✓	(✓)		✓	✓	✓	+++	+++	++	++
Hessian-Affine	(✓)	✓		✓	✓	✓	+++	+++	+++	++
Salient Regions	(✓)	✓		✓	✓	(✓)	+	+	++	+
Edge-based	✓			✓	✓	✓	+++	+++	+	+
MSER			✓	✓	✓	✓	+++	+++	++	+++
Intensity-based			✓	✓	✓	✓	++	++	++	++
Superpixels			✓	✓	(✓)	(✓)	+	+	+	+

Key reference: <http://www.robots.ox.ac.uk/~vgg/research/affine/>

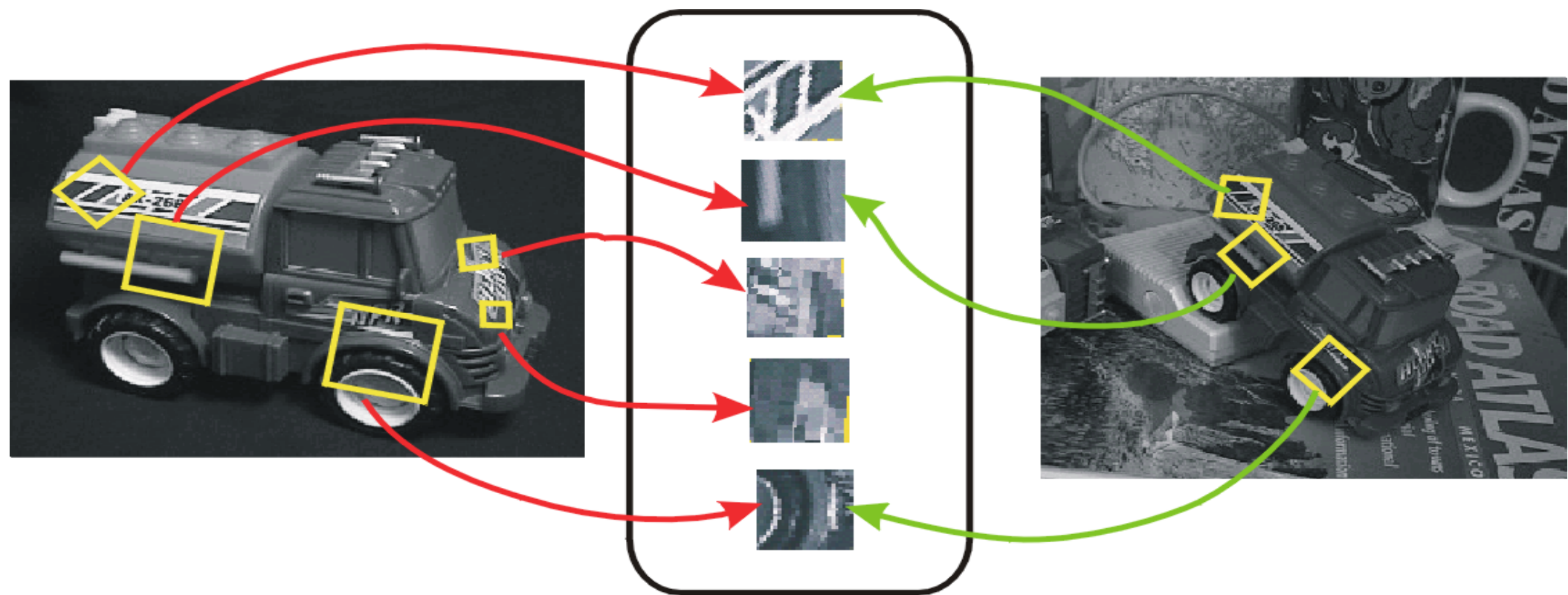
From feature detection to description

- Scaled & rotated versions of the same neighborhood will give rise to blobs that are related by the same transformation
- What to do if we want to compare the appearance of these image regions?
 - Normalization: transform the regions into same-size circles
 - Problem: **rotational ambiguity**



Invariant local features

- Image content is transformed into local features that are invariant to *translation*, *rotation*, *scale*, and other imaging parameters



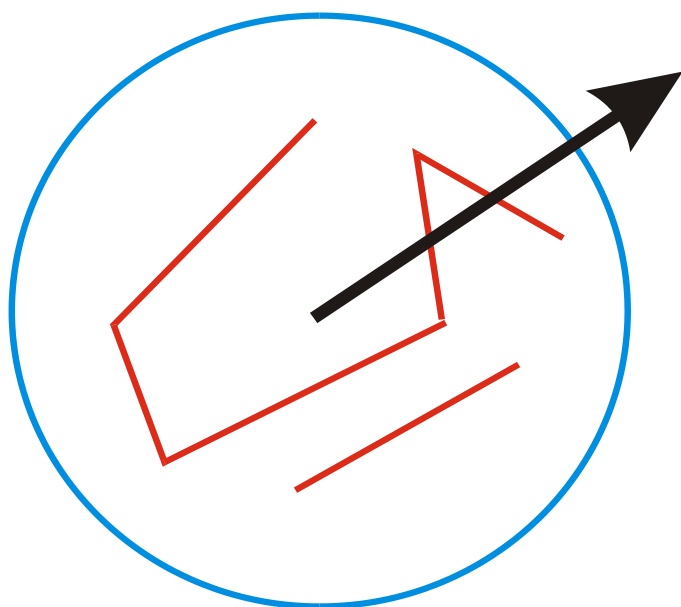
Reference: CVPR 2003 Tutorial on “Recognition and Matching Based on Local Invariant Features” by David Lowe

Desired properties of feature descriptors

- Easily compared (compact, fixed-dimensional)
- Invariant:
 - Translation
 - Rotation
 - Scale
 - Change in image brightness (illumination)
 - *Change in perspective?*

SIFT descriptors

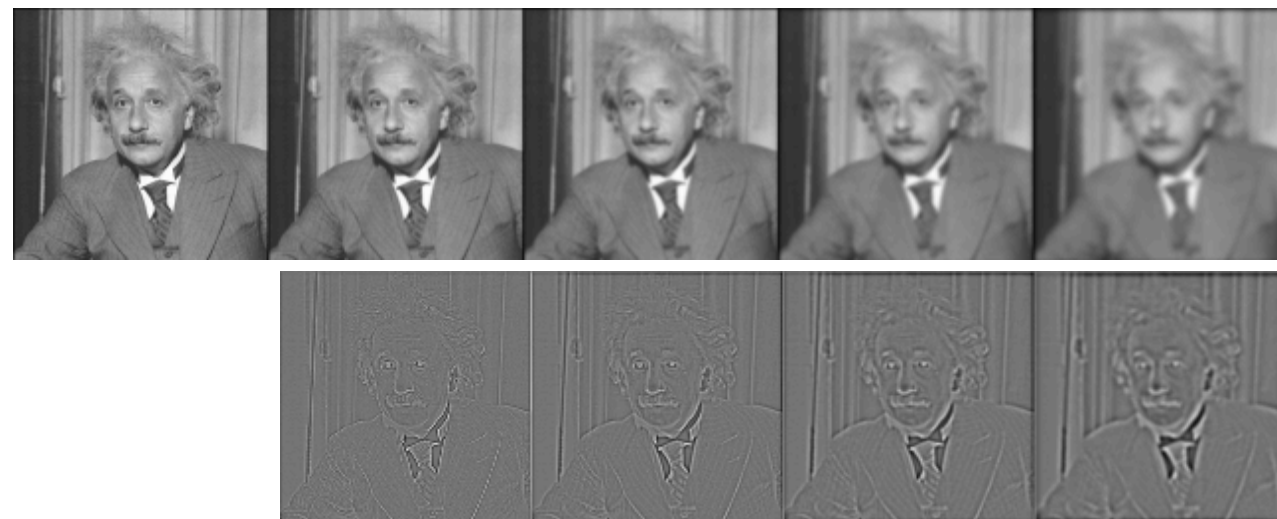
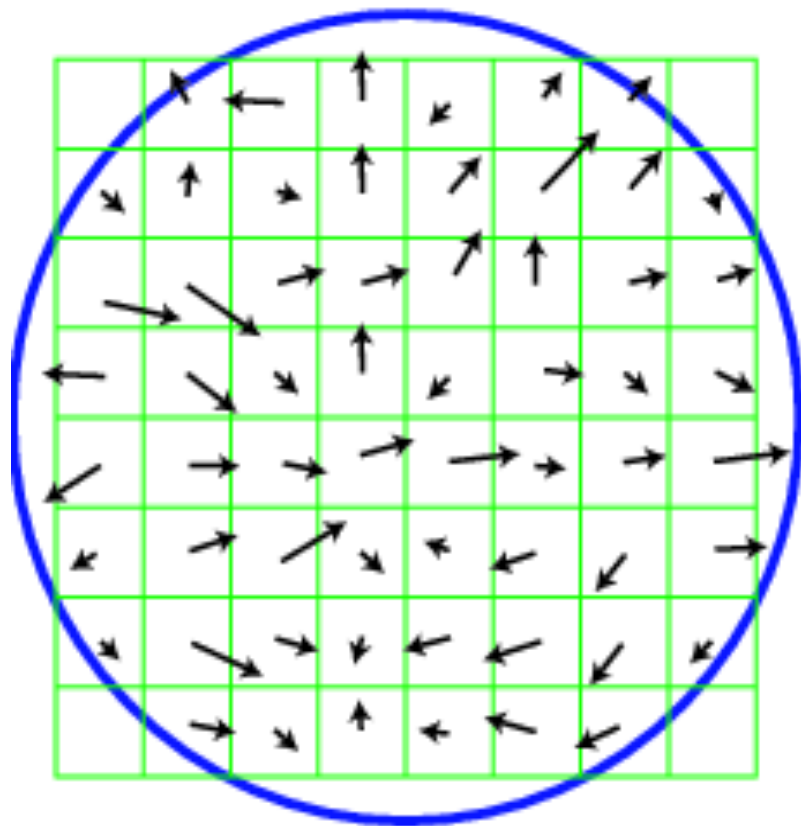
- We are given a keypoint and its scale from DoG
- To assign the orientation to circular image windows:
 - Create histogram of local gradient directions in the patch
 - Then we select a *characteristic orientation* for the keypoint based on the most prominent gradient (see next slides)
 - We will describe all features relative to this orientation



If the keypoint appears rotated in another image, the features will be the same, because they're relative to the characteristic orientation

SIFT descriptor formation

- Use the blurred image associated with keypoint's scale
- Take image gradients over the keypoint neighborhood
- To become rotation invariant, rotate the gradient directions AND locations by keypoint orientation

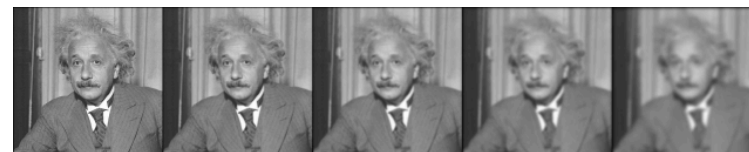
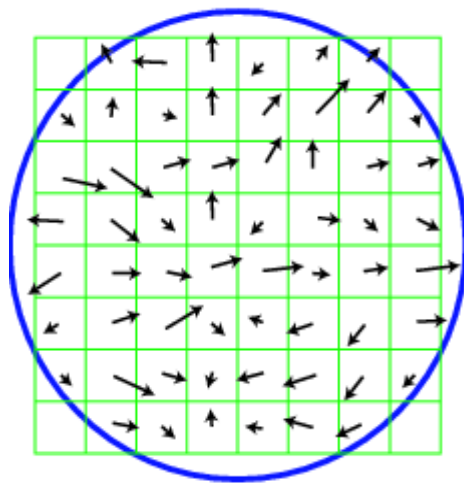


$L(x, y, \sigma)$

$D(x, y, \sigma)$

SIFT descriptor formation

- Use the blurred image associated with keypoint's scale
- Take image gradients over the keypoint neighborhood
- To become rotation invariant, rotate the gradient directions AND locations by keypoint orientation



$L(x, y, \sigma)$



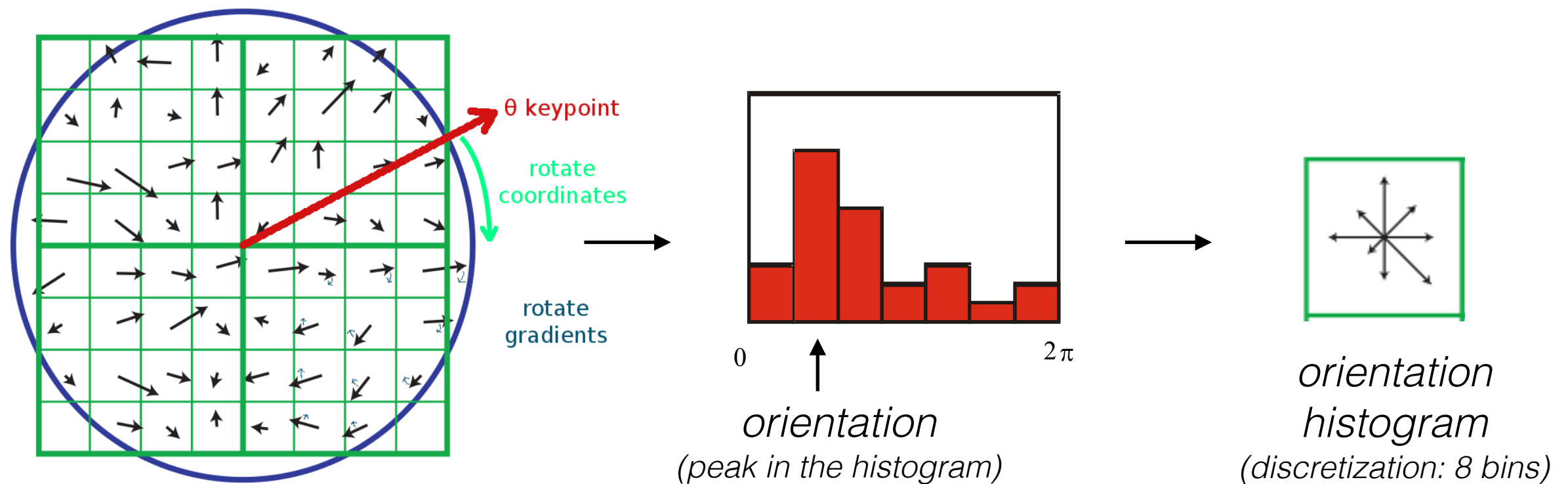
$D(x, y, \sigma)$

$$m(x, y) = \sqrt{(L(x+1, y) - L(x-1, y))^2 + (L(x, y+1) - L(x, y-1))^2}$$

$$\theta(x, y) = \text{atan2}(L(x, y+1) - L(x, y-1), L(x+1, y) - L(x-1, y))$$

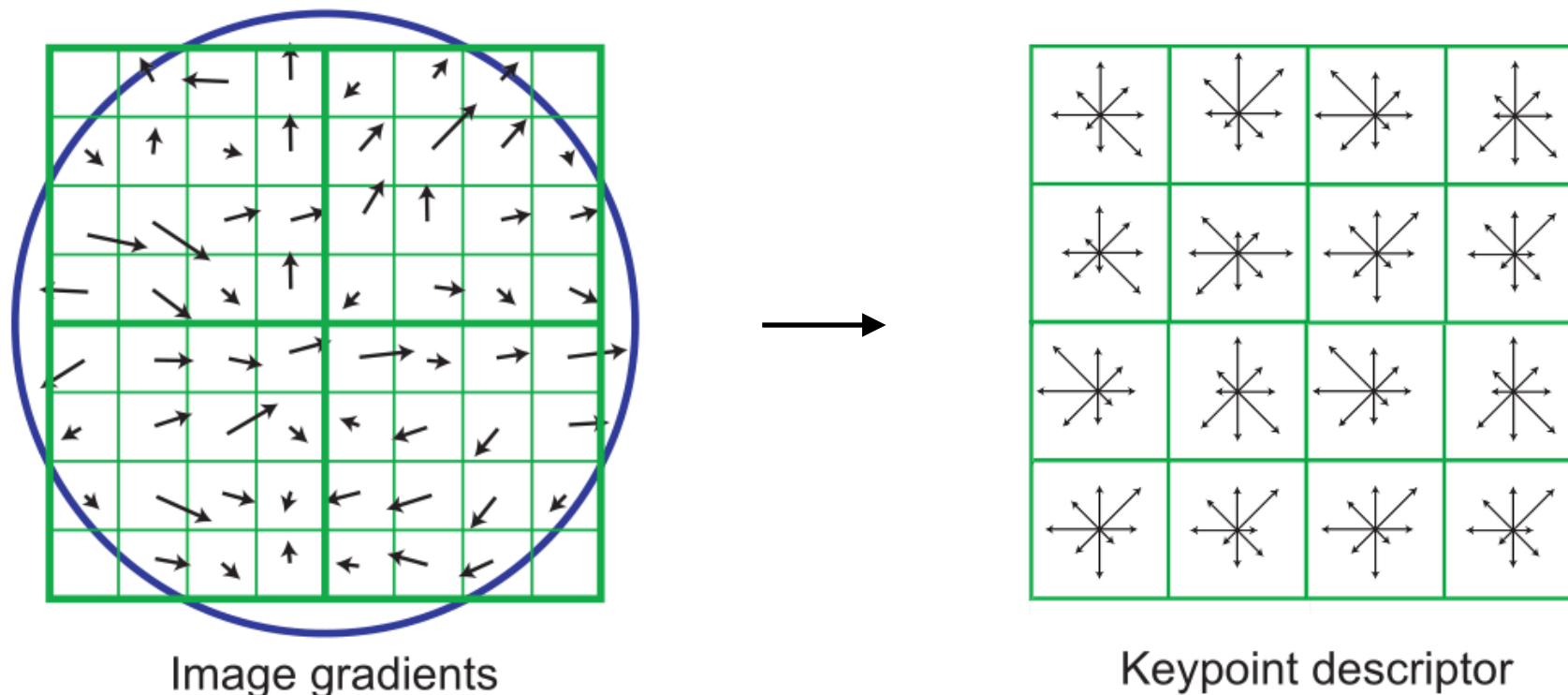
SIFT descriptor formation

- Use the blurred image associated with keypoint's scale
- Take image gradients over the keypoint neighborhood
- To become rotation invariant, rotate the gradient directions AND locations by keypoint orientation



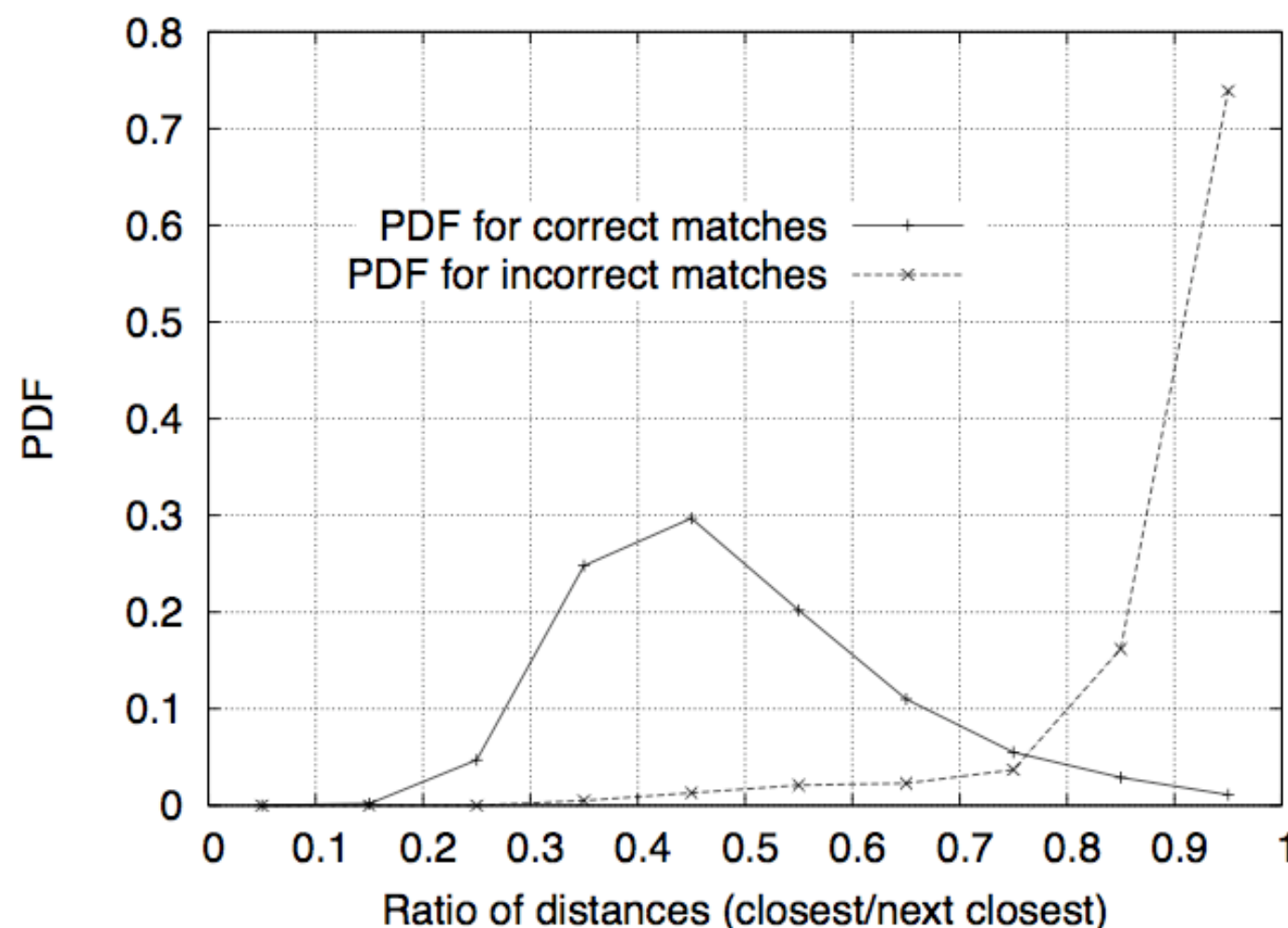
SIFT descriptor formation

- 8 orientation bins per histogram and a 4x4 histogram array yields $8 \times 4 \times 4 = 128$ numbers
- So a SIFT descriptor is a 128-d vector invariant to rotation (because we rotated the descriptor) and scale (thanks to DoG)
- We can now compare each vector from image *A* to each vector from image *B* to find matching keypoints



SIFT matching

- Euclidean distance between SIFT descriptor gives a good measure of keypoint similarity
- Ratio of distances is more reliable for matching



The probability that a match is correct can be determined by taking the ratio of distance from the closest neighbor to the distance of the second closest
(plot: PDF obtained using a database of approx. 40,000 keypoints)

SIFT matching

- An example:



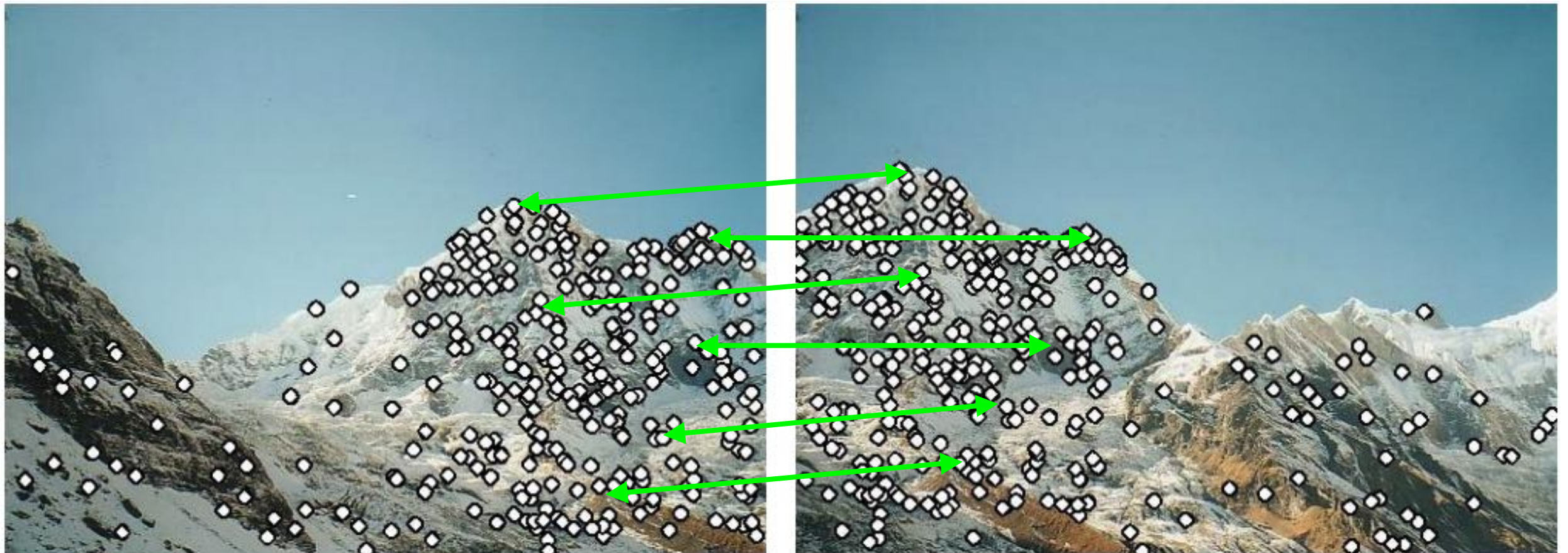
Image matching: main flow

- Image matching with local visual features (SIFT)
 - Common pipeline: three (plus one) steps



Image matching: main flow

- Image matching with local visual features (SIFT)
 - Common pipeline: three (plus one) steps



Step 1: detect keypoints

Step 2: build keypoints descriptors

Step 3: match keypoint features

Step 4: align images (i.e. fitting the transformation)

Image matching: main flow

- Image matching with local visual features (SIFT)
 - Common pipeline: three (plus one) steps



Step 1: detect
keypoints

Step 2: build
keypoints descriptors

Step 3: match
keypoint features

Step 4: align images (i.e.
fitting the transformation)

Image matching: main flow

- Image matching with local visual features (SIFT)
 - ▶ “Additional step”: image alignment



Step 1: detect
keypoints

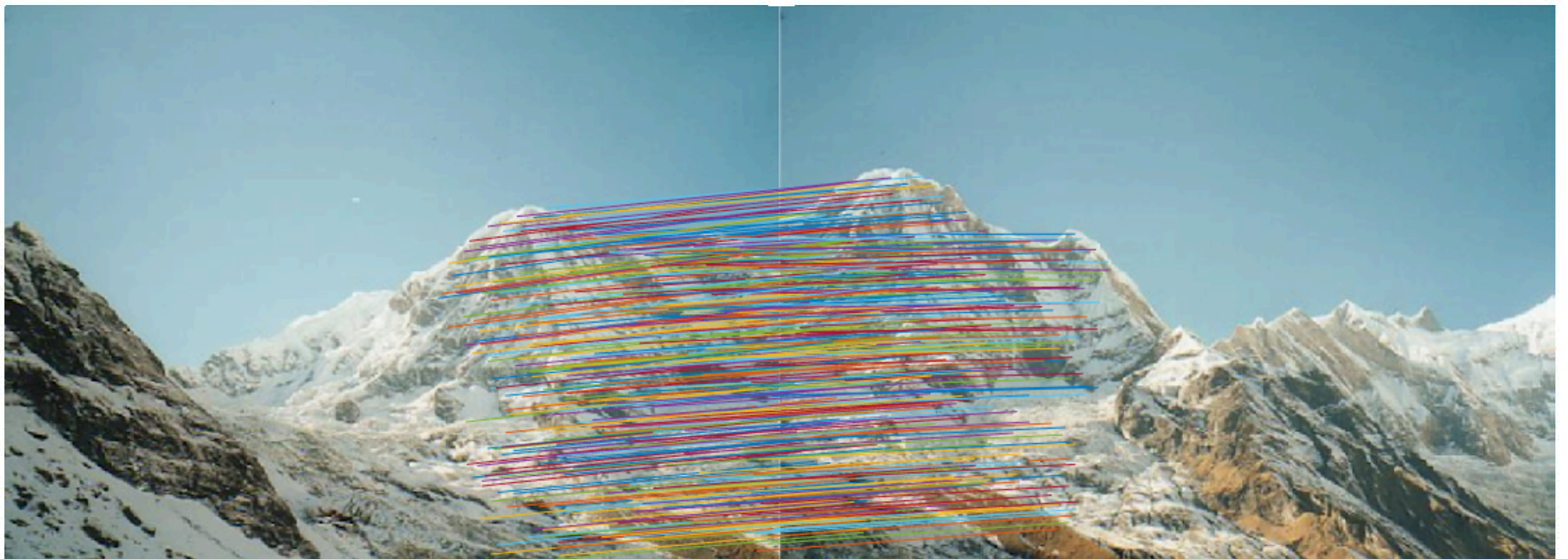
Step 2: build
keypoints descriptors

Step 3: match
keypoint features

Step 4: align images (i.e.
fitting the transformation)

Geometric verification

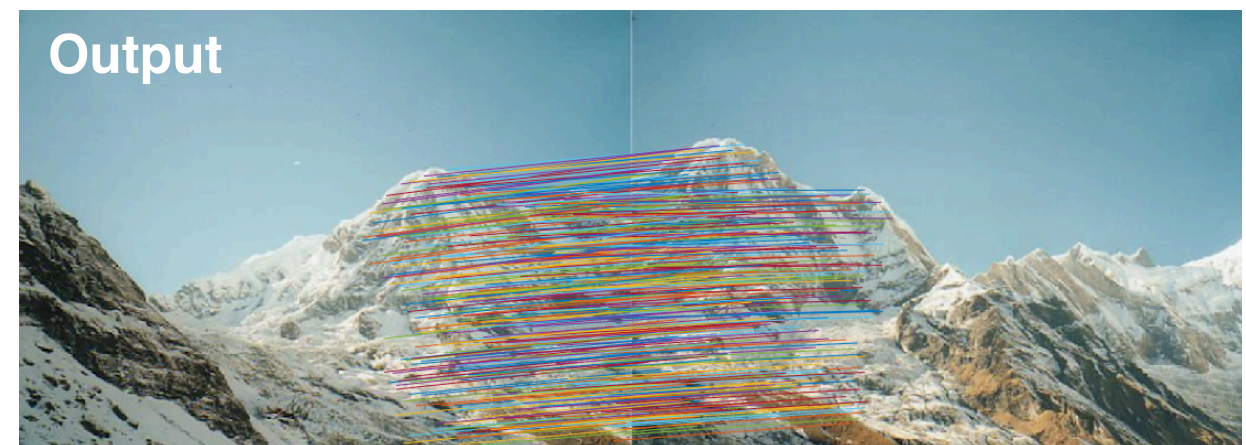
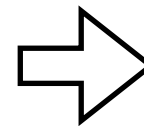
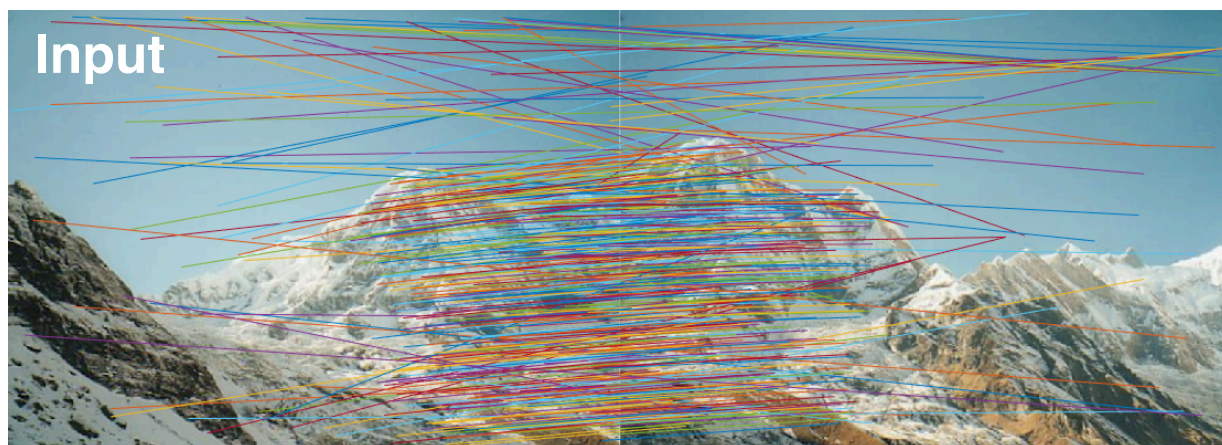
- A bit more on “step 4”, i.e. *geometric verification*
 - The idea is to test whether matches are consistent with an overall image transformation
 - Inconsistent matches are rejected



RANSAC: matching robust to outliers

(RANdom SAmple Consensus)

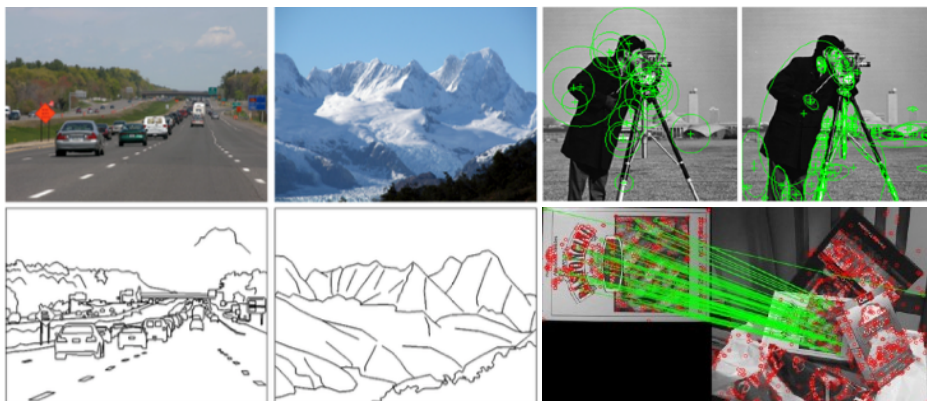
- **Input:** M tentative features matches $(x_1, x_1'), \dots, (x_M, x_M')$
- **Output:** (affine) transformation (A^*, T^*) with the largest number of inlier matches



1. Repeat a large number of times:
 - Randomly sample a **minimal subset** of matches sufficient to estimate (A, T)
 - Find **inliers**, i.e. other matches that are compatible with (A, T)
2. Return (A^*, T^*) as the pair (A, T) with the largest number of inliers

Summary

- Edge detection
- Image gradients
- Local invariant features
- DoG and SIFT descriptors



Background reading:
Forsyth and Ponce, Computer Vision, Chapter 8
D.G. Lowe, SIFT paper, IJCV 2004

Coming up

- **Two labs on local features:**
 - Lab on visual features and image matching (+ contest!)
- **Then, we will move on to the next “chapter”:**
 - Introduction to visual recognition, image classification

